

江苏师范大学

Practical training report

实 验 名 称： **Microcontroller Training**

姓　　名： SSAMULA PHILLIP PETER

学　　院： ELECTRICAL ENGINEERING AUTOMATION

专　　业： 电气工程

指 导 教 师： Huang Huan

TABLE OF CONTENTS

I. PROJECT INTRODUCTION 2

II. EXPERIMENT 1: USE OF KEIL C 4

A. Software Introduction and Installation 4
B. Build a Project 4
C. Program Debugging 6

III. EXPERIMENT 2: USE OF PROTEUS 8

A. Software Introduction and Installation 8
B. Build a Project 8
C. Simple Assembly / Code Testing (Experimental Results) 10

IV. EXPERIMENT 3: TEMPERATURE AND HUMIDITY DETECTION AND ALARM SYSTEM 12

A. Experiment Contents 12
B. Hardware Design 14
C. Software Design 14
D. Experimental Results 16

V. EXPERIMENT 4: ULTRASONIC DISTANCE MEASUREMENT 18

A. Experiment Contents 18
B. Hardware Design 18
C. Software Design 20
D. Experimental Results 20

VI. EXPERIMENT 5: BRUSHLESS DC MOTOR (BLDC) WITH POSITION SENSOR PWM 22

A. Experiment Contents 22
B. Hardware Design 24
C. Software Design 24
D. Experimental Results 26

VII. REPORT CONCLUSION 28

VIII. EXPERIMENT CODE IN FULL 30

A. Experiment 1 Code: Electronic Clock Design 30
B. Experiment 3 Code: Temperature and Humidity Detection 32
C. Experiment 4 Code: Ultrasonic Distance Measurement 34
D. Experiment 5 Code: PIC BLDC Control 36

PROJECT INTRODUCTION

This comprehensive practical training report documents the systematic design, software compilation, and hardware simulation of various microcontroller-based embedded systems. The primary objective of this project is to bridge theoretical microcontroller principles with practical Electronic Design Automation (EDA) workflows. By utilizing industry-standard tools such as Keil µVision, MPLAB X IDE, and the Proteus Design Suite, this report explores the complete lifecycle of embedded engineering—from raw C-code logic to dynamic electromechanical simulation.

To systematically test and evaluate these concepts, the report is structured into five distinct experimental sections:

- **Section 1 (Experiment ONE - Use of Keil C):** Located at the beginning of the report, this foundational section explores the software engineering environment. Utilizing an *Electronic Clock Design (8051)* as a practical testbed, the experiment aims to establish the methodologies for writing, building, and debugging embedded C code, ultimately testing the generation of executable machine logic (.hex files).
- **Section 2 (Experiment TWO - Use of Proteus):** This section transitions into Virtual System Modelling (VSM). It aims to explore the dependency between programmed logic and physical circuit behavior. By utilizing schematic capture tools, this experiment tests the importation of compiled code into a virtual microcontroller to validate dynamic hardware responses.
- **Section 3 (Experiment THREE - Temperature and Humidity Detection and Alarm System):** Located in the third section, this core project aims to explore digital data acquisition and closed-loop control logic. By interfacing the 8051 microcontroller with a DHT11 sensor, the experiment tests the system's ability to decode 1-Wire communication protocols and autonomously actuate hardware alarms based on user-defined environmental thresholds.
- **Section 4 (Experiment FOUR - Ultrasonic Distance Measurement):** This experiment investigates spatial measurement via acoustic wave propagation. The aim is to test time-of-flight principles by utilizing the microcontroller's internal 16-bit hardware timers. The system explores the mathematical conversion of microsecond-level echo delays into precise, dynamic floating-point distance metrics.
- **Section 5 (Experiment FIVE - Brushless DC Motor (BLDC) with Position Sensor PWM):** Located in the final experimental section, this capstone project transitions to a PIC microcontroller architecture. It aims to test advanced electromechanical control concepts. Specifically, it explores the decoding of 3-bit Hall effect sensor feedback to achieve 6-step electronic commutation, alongside the implementation of software-driven Pulse Width Modulation (PWM) for real-time motor speed regulation.

1. EXPERIMENT 1: USE OF KEIL C

a. Software Introduction and Installation

Project name: 1、基于 51 单片机的电子时钟设计 (Electronic Clock Design Based on 51 Microcontroller)

Keil µVision (specifically version 4, as utilized in this training) is an industry-standard Integrated Development Environment (IDE) designed for writing, compiling, and debugging embedded software for the 8051 microcontroller architecture.

- **Aim & Principle:** The sub-objective of this section is to write and compile the C-language logic required to drive a digital clock. This involves initializing a DS1302 Real-Time Clock (RTC) chip and configuring hardware timers.
- **Equipment:** A personal computer running the Windows operating system and Keil µVision 4 IDE.

Installation and Initialization Process: The setup of Keil µVision 4 was completed via a standard wizard-driven installation process. The executable installer was launched, and the target directory was carefully selected to ensure the file path contained no non-English characters (a critical step to prevent future compilation or linking errors). Once the core IDE components were extracted and installed, the application was launched to reveal the initial workspace. This default graphical user interface is structured with a standard top menu bar for tool selection, a Project navigation pane docked to the left for file management, and a central text editor pane ready for the initialization of a new embedded C project.**b. Build a Project**

The software design phase begins by initializing a new project targeting the AT89C52 microcontroller. The foundational C code—handling the DS1302 data retrieval and 7-segment display multiplexing—is written into the main editor window.

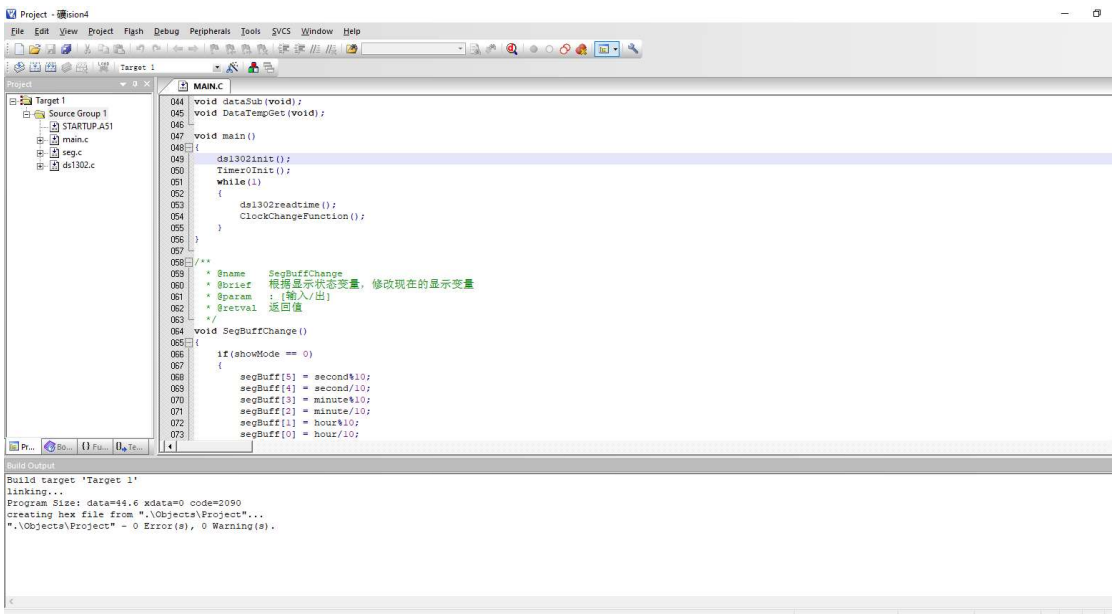


Figure 1.3: The Keil C editor displaying the `main()` function and the initial calls to the DS1302 and Timer0 hardware.

NOTE: *(For the complete source code of this experiment, see Section VIII.A, p. 30) The complete source code for this specific experiment is attached in the Appendix at the end of this document under "Experiment 1 Code: Electronic Clock Design."*

Once the logic is written, the code must be translated into machine-readable instructions. Before building, the "Options for Target" menu is accessed to ensure the "Create HEX file" configuration is enabled. This step is absolutely critical; without this .hex file, the Proteus simulation software (covered in Experiment 2) will have no executable instructions to run, rendering the virtual hardware useless.

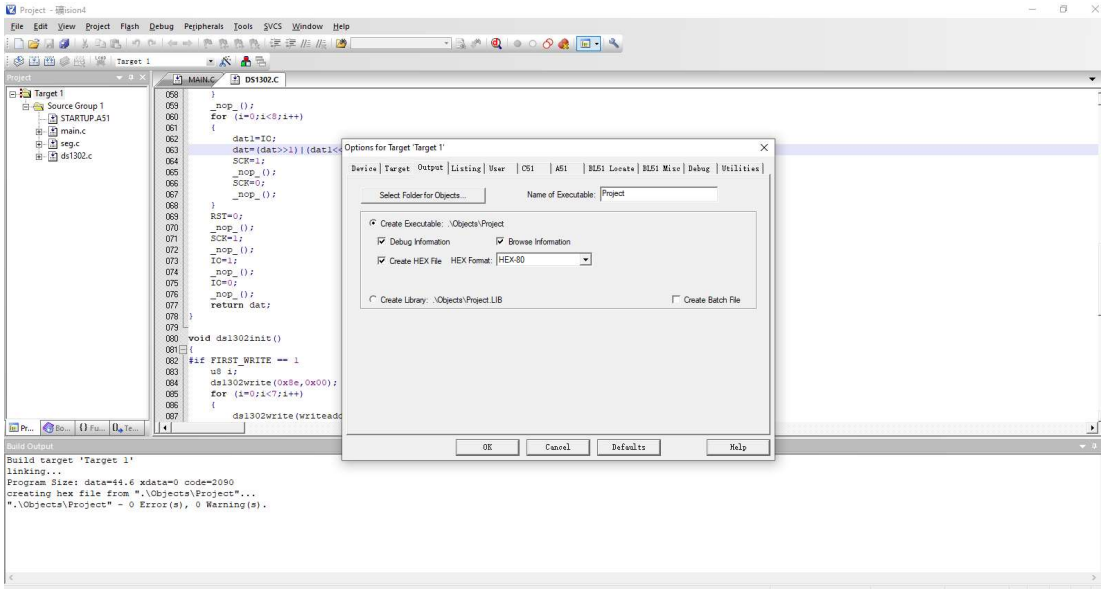


Figure 1.4: Configuring the build target to output an executable HEX file.

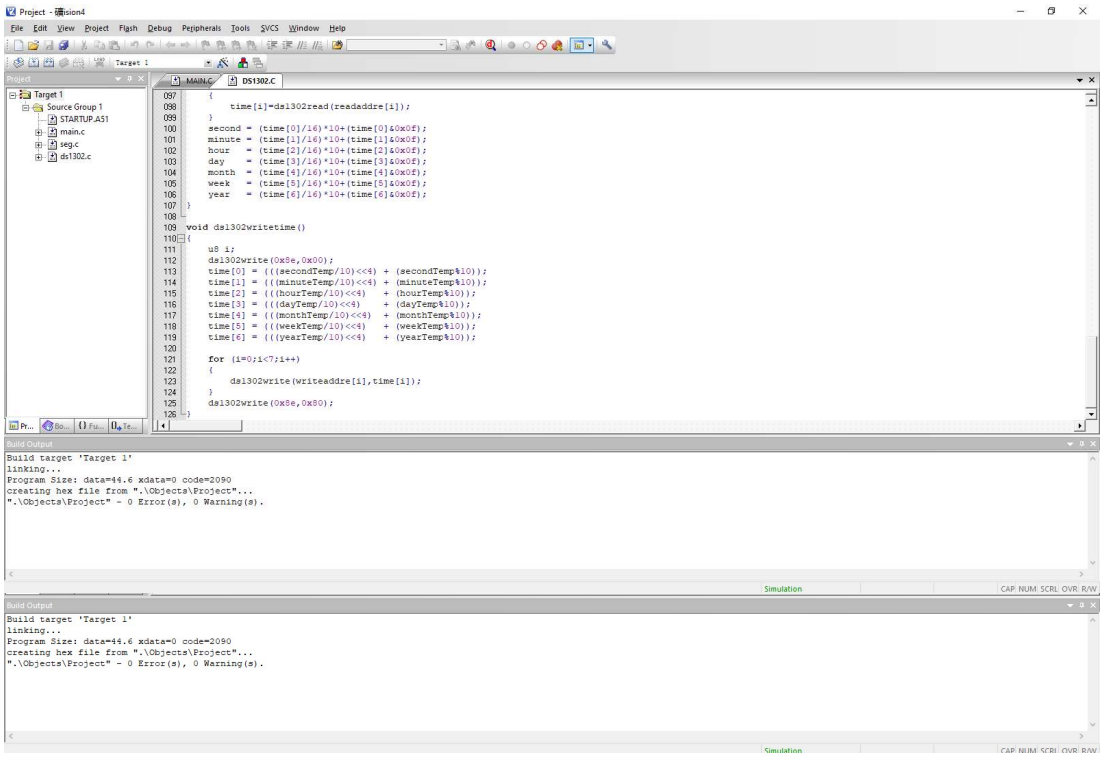


Figure 1.5: The successful compilation process translating C code into machine logic.

c. Program Debugging

Before porting the software to the hardware simulator, Keil's internal VSM debugging mode is utilized. Debugging allows the programmer to step through the code line-by-line or view assembly language translations to ensure memory addresses and CPU registers are functioning as intended.

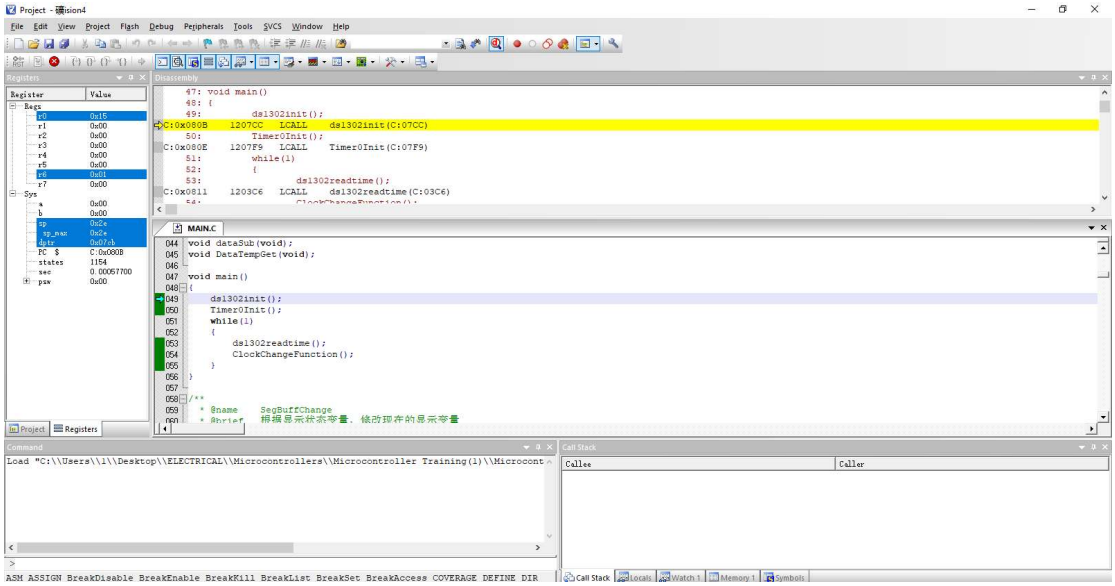


Figure 1.6: Keil operating in Debug/Disassembly mode, allowing for register-level inspection of the clock initialization routines.

2. EXPERIMENT 2: USE OF PROTEUS

2.1 Software Introduction and Installation

Project name: 1、基于 51 单片机的电子时钟设计 (Electronic Clock Design Based on 51 Microcontroller)

Proteus Design Suite (specifically Proteus Professional 9) is an Electronic Design Automation (EDA) tool equipped with a mixed-mode SPICE simulation engine. It allows engineers to draw electrical schematics and virtually test microcontroller-based designs without soldering physical components.

- **Aim & Principle:** The goal is to simulate the physical hardware of the electronic clock. The core principle tested here is display multiplexing using 74HC138 and 74HC573 logic ICs to drive an 8-digit 7-segment display (7SEG-MPX8-CC-BLUE), alongside SPI-like communication with the DS1302 RTC.
- **Equipment (Virtual):** AT89C52, DS1302 RTC, 74HC138 Decoder, 74HC573 Latch, 7-Segment Display, Resistors, and Capacitors.

Installation and Initialization Process: The Proteus Professional 9 software was installed utilizing the standard Labcenter Electronics setup wizard. Following the configuration of the local license key and default component library directories, the application was initialized. The default ISIS (Intelligent Schematic Input System) workspace opens to a central, gridded canvas designated for hardware design. Docked to the left is the component bin and mode selector toolbar, which allows the user to easily search the database, retrieve virtual microcontrollers or logic ICs, and place them into the simulation environment for wiring.

2.2 Build a Project

The hardware components were selected from the Proteus library and wired together in the schematic capture window. However, hardware alone cannot function without software.

As an explicit demonstration of the dependency between Keil and Proteus, an attempt was made to run the simulation before attaching the HEX file generated in Experiment 1. The Proteus simulation engine immediately halted, proving that the AT89C52 requires external programmed logic to operate.

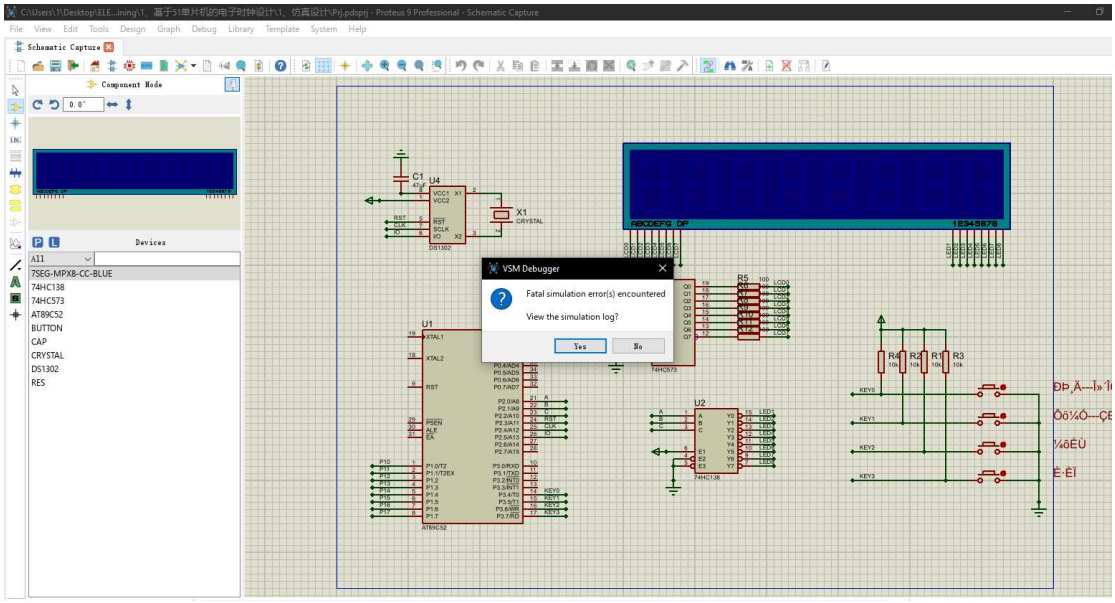


Figure 2.3: Proteus displaying a "Fatal simulation error(s) encountered" popup due to a missing program file.

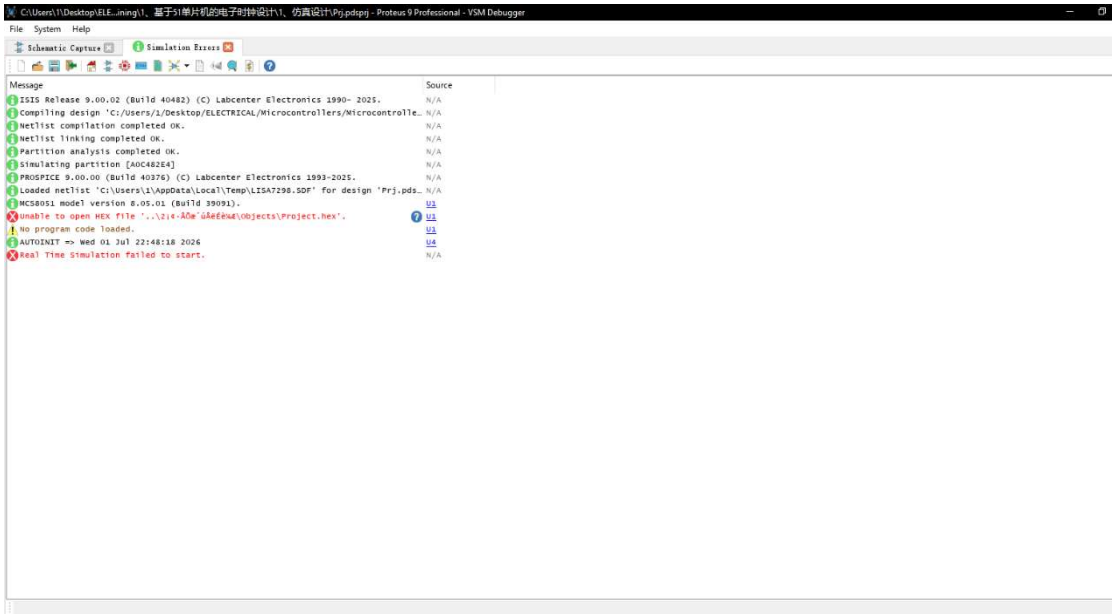


Figure 2.4: The Proteus simulation log explicitly stating "Unable to open HEX file" and "No program code loaded."

To resolve this, the properties of the AT89C52 component were accessed (by double-clicking the IC), and the 'Program File' path was linked directly to the `.hex` file successfully generated by Keil in Section 1.

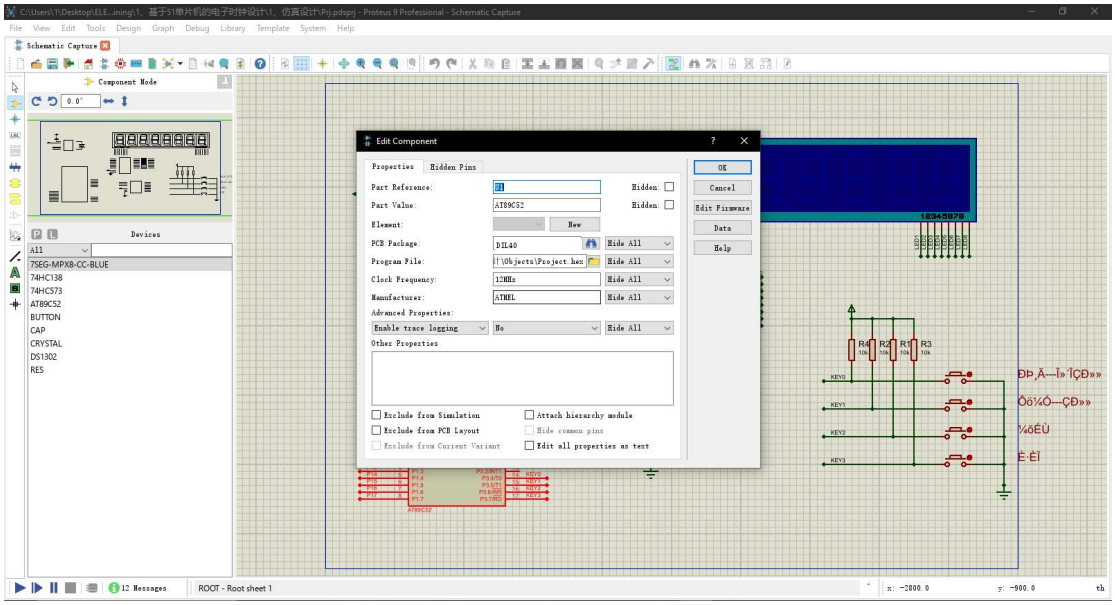


Figure 2.5: Importing the compiled Keil executable into the virtual microcontroller.

2.3 Simple Assembly / Code Testing (Experimental Results)

With the Keil software and Proteus hardware finally linked, the simulation was executed. The results were highly successful.

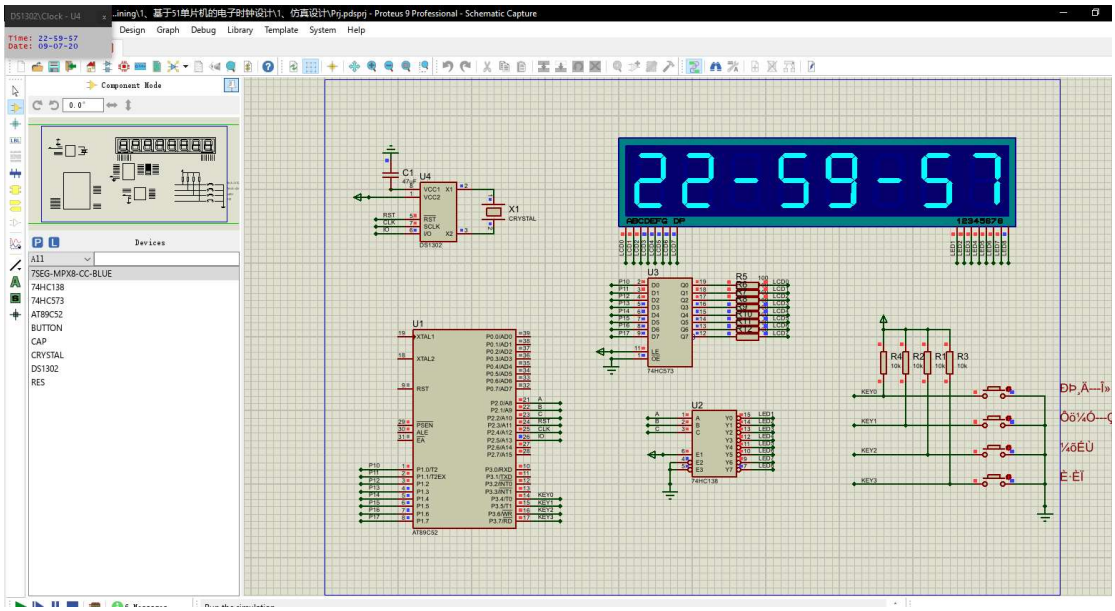


Figure 2.6: The simulation running successfully, decoding the DS1302 data to display an initial time of 22-59-57.

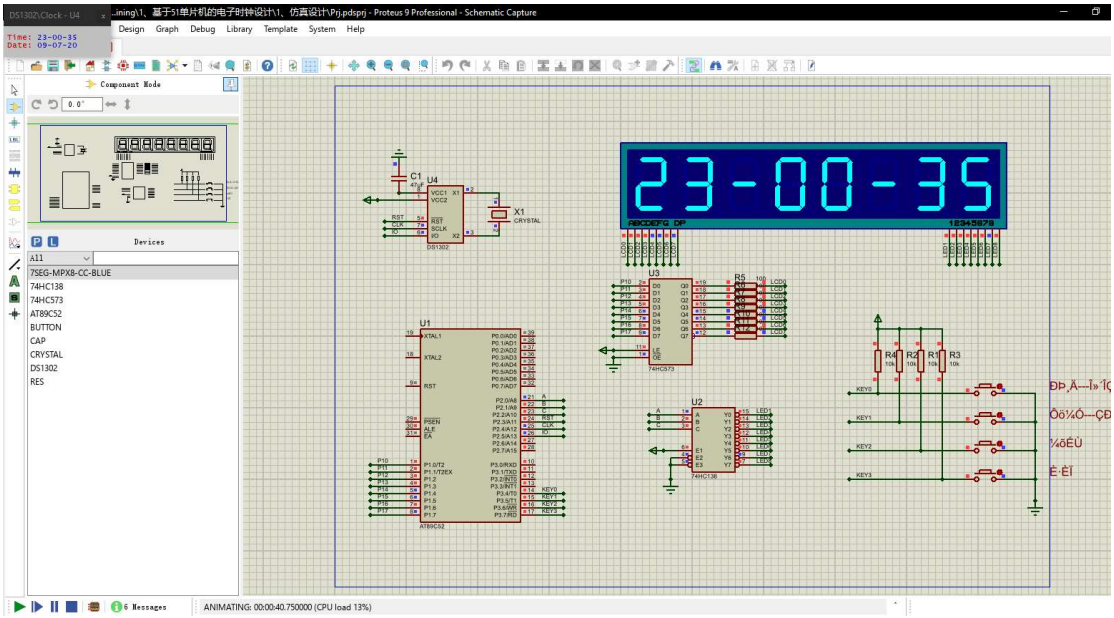


Figure 2.7: The simulation progressed forward in time, showing the rollover to 23-00-35, proving continuous dynamic operation.

Record of Observations and Conclusion: Upon clicking "Run Simulation" with the correct HEX file attached, the blue 7-segment displays immediately illuminated. The virtual DS1302 maintained time accurately, and the 74HC138/573 logic successfully multiplexed the data across all 8 digits without visible flickering. By observing the time progress dynamically from 22:59:57 to 23:00:35, it is confirmed that the embedded logic is functioning continuously.

Overall Conclusion (Experiments 1 & 2):

These two foundational sections successfully demonstrate that embedded systems engineering is a codependent process. Keil µVision is strictly required to translate human-readable C code into machine logic, while Proteus is required to validate how that logic interacts with physical electrical pathways. By successfully building and simulating the Electronic Clock, proficiency in this dual-software workflow has been fully established.

3. Experiment THREE: 4. 基于 51 单片机的温湿度检测报警系统

(Temperature and Humidity Detection and Alarm System Based on 51 Microcontroller)

a. Experiment Contents

1. Aim

The primary objective of this experiment is to design, program, and simulate an environmental monitoring system capable of measuring real-time temperature and humidity using an AT89C52 microcontroller and a DHT11 sensor. The system aims to display these environmental parameters on an LCD1602 screen, transmit the data serially to a virtual terminal for remote monitoring, and utilize a threshold-based logic algorithm to trigger localized hardware alarms (LEDs) when environmental conditions deviate from user-defined safety parameters.

2. Principle

This system operates on the principles of digital data acquisition, serial communication, and bang-bang control theory.

- **Data Acquisition (DHT11 Protocol):** The DHT11 sensor utilizes a specialized single-bus digital signal technology. It combines a resistive-type humidity measurement component and an NTC (Negative Temperature Coefficient) temperature measurement component. Instead of outputting an analog voltage that requires an ADC, the DHT11 communicates with the microcontroller via a precise 1-Wire protocol. The microcontroller sends a start signal by pulling the data line low for at least **18 ms**, prompting the DHT11 to respond with a 40-bit data packet containing the integral and decimal values of both temperature and humidity, alongside an 8-bit checksum for data validation.
- **Display Multiplexing:** The LCD1602 utilizes parallel communication. Because the 8051's Port 0 (P0) operates as an open-drain output, an external pull-up resistor network is required to provide the necessary high-level voltage (**V_CC**) to drive the data pins (D0-D7) of the LCD.
- **Threshold Control Logic:** The alarm system employs a simple closed-loop logic structure. The microcontroller continuously compares the real-time variables (**T_current** and **H_current**) against predefined upper and lower boundaries (**T_high**, **T_low**, **H_high**, **H_low**). If a boundary is breached, the microcontroller alters the logic state of specific GPIO pins to actuate the alarm indicators.

3. Equipment

Hardware Design (Real and Simulated):

- **Microcontroller:** AT89C52 (8051 Architecture).
- **Sensor:** DHT11 Digital Temperature and Humidity Sensor.
- **Display:** LM016L (Standard 16x2 Character LCD).
- **Indicators:** 4 × Yellow LEDs representing distinct alarm states.
- **Inputs:** 4 × Push-button switches for threshold parameter adjustment.
- **Communication:** RS-232 Virtual Terminal interface.
- **Passive Components:** 11.0592 MHz Crystal Oscillator, 30 pF Capacitors, 10 µF Electrolytic Capacitor, 100 Ω Resistors, and an 8-pin Resistor Pack (RESPACK-8).

Software Design:

- Keil µVision IDE for embedded C compilation.
- Proteus VSM for mixed-mode simulation.
- Core software modules include: `Timer0_Init()` for timing, `UartInit()` for serial communication, and `CheckLimits()` for alarm actuation.

b. Hardware Design

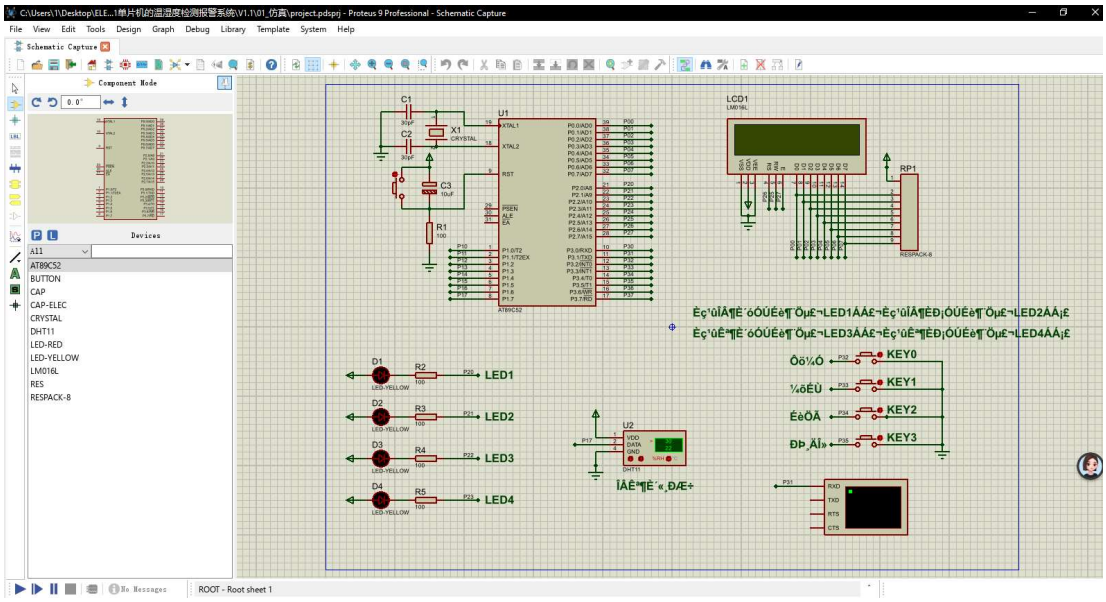


Figure 3.1: Complete Proteus wiring schematic of the environmental monitoring system prior to simulation execution.

Detailed Explanation of Components and Wiring Configuration:

The hardware architecture is meticulously designed around the AT89C52 microcontroller (U1), which orchestrates the flow of data between the input sensors and output peripherals.

- **Microcontroller Core & Clock Circuit:** The AT89C52 is driven by an external crystal oscillator (X1) rated at **11.0592 MHz**. This specific frequency is critical for embedded systems utilizing UART, as it perfectly divides down to generate standard baud rates (e.g., 9600 bps) without timing errors. The reset circuit consists of a capacitor (C3) and resistor (R1) connected to pin 9 (RST) to ensure the microcontroller starts from a known state upon power-up.
- **Sensor Integration (U2 - DHT11):** The DHT11 module is wired to receive power (V_DD) and Ground. Its bidirectional DATA pin is routed to the microcontroller's Port 1.7 (P1.7). A pull-up resistor is conceptually required on this data line to keep the bus high when idle, ensuring the 1-Wire protocol functions correctly.
- **LCD Interface (LCD1):** The LM016L display is interfaced in 8-bit mode. The data bus (D0-D7) is connected directly to Port 0 (P0.0 - P0.7). Because Port 0 lacks internal pull-ups, an external resistor network (RP1, RESPACK-8) is wired to the port to pull the lines to logic HIGH, ensuring the LCD registers the data correctly. The control pins RS, RW, and E are managed by Port 2 pins (P2.5, P2.6, and P2.7).
- **Alarm Actuators (LEDs):** Four distinct alarm states are represented by D1-D4 (LED-YELLOW). They are wired to Port 2 (P2.0 to P2.3) via 100 Ω current-limiting resistors (R2-R5). The system is designed with an active-low configuration; the microcontroller must output a logic 0 to sink current and illuminate the LED.
    - **LED1 (P2.0):** Reduce Temperature Alarm
    - **LED2 (P2.1):** Add Temperature Alarm
    - **LED3 (P2.2):** Reduce Humidity Alarm
    - **LED4 (P2.3):** Add Humidity Alarm
- **User Input Interface (Buttons):** Four tactile switches (KEY0-KEY3) are tied to Port 3 (P3.2 to P3.5). The other side of each switch is connected to common ground. When pressed, they pull the respective microcontroller pin low, allowing the user to navigate the UI page and adjust the `hum_high`, `hum_low`, `temp_high`, and `temp_low` parameters.
- **Serial Terminal:** The Virtual Terminal is cross-wired to the microcontroller's UART pins; the MCU's TXD (P3.1) connects to the terminal's RXD, and the MCU's RXD (P3.0) connects to the terminal's TXD.

c. Software Design

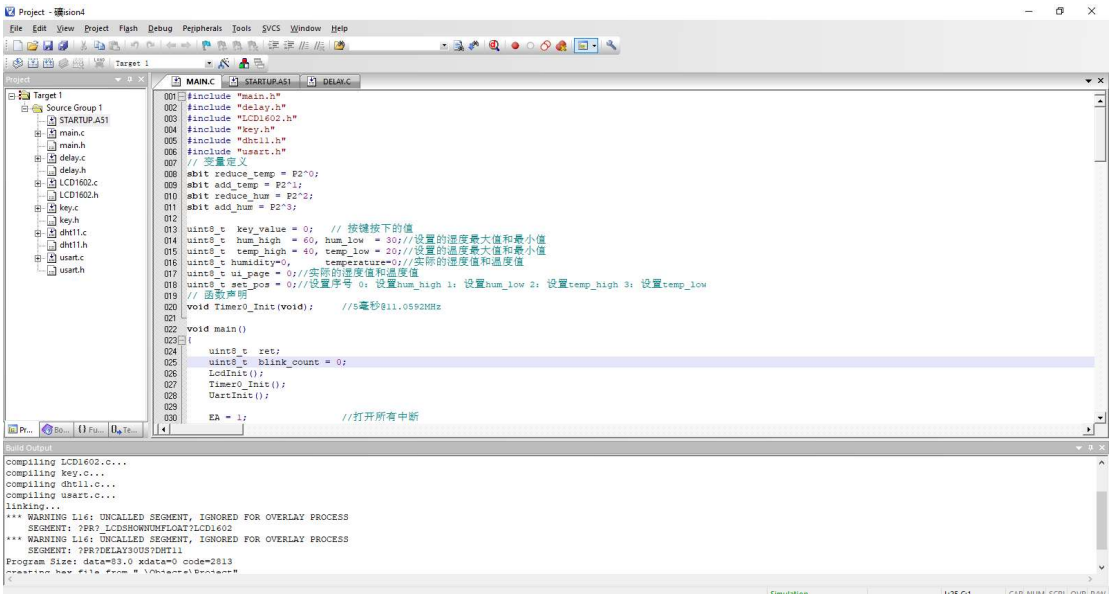
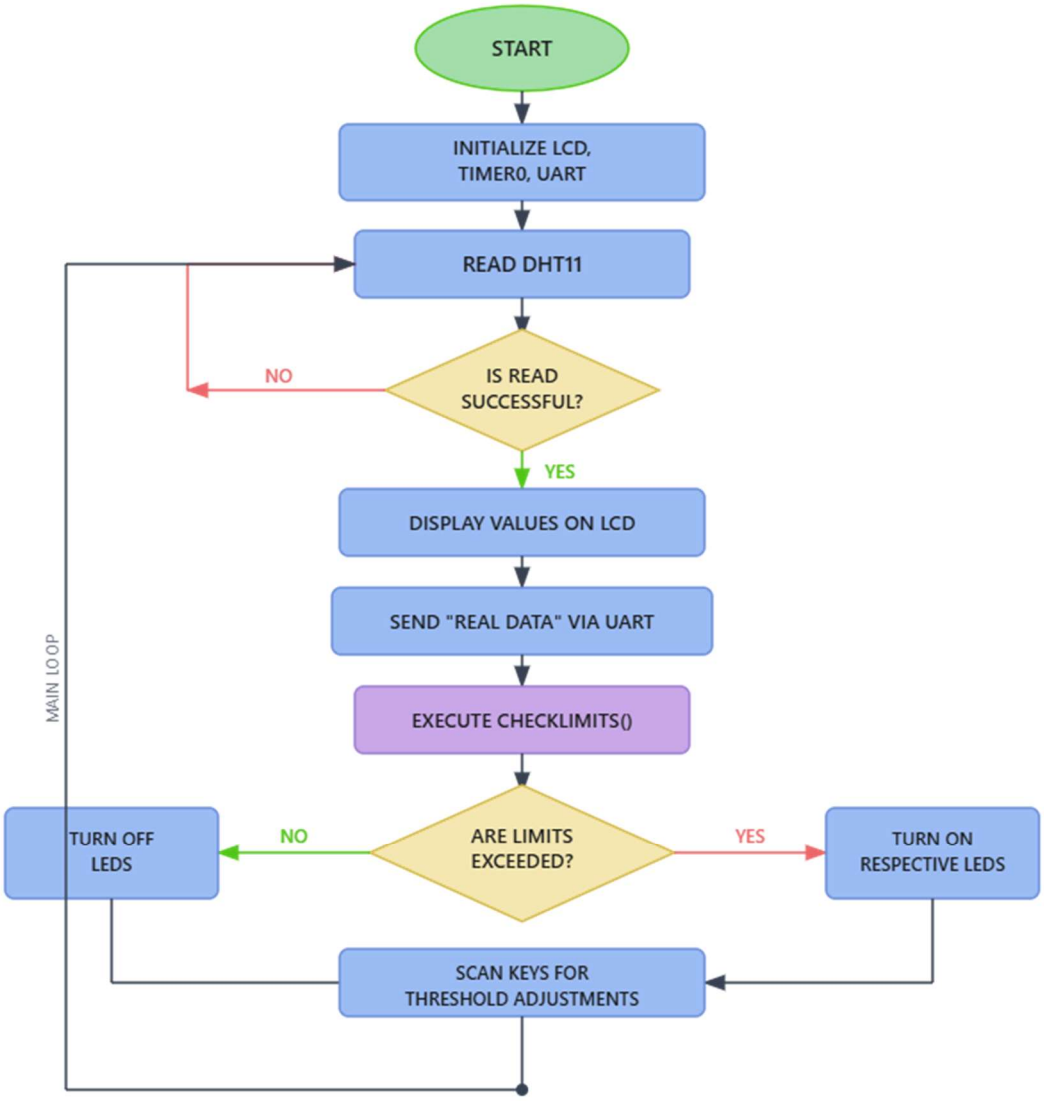


Figure 3.2: Keil IDE workspace showing the main.c file, global variable definitions, and initialization routines and Software logic for the threshold checking and alarm triggering mechanism.

NOTE: *(For the complete source code of this experiment, see Section VIII.B, p. 32) The complete, unabridged C source code for this experiment is located in the Appendix at the end of this document under "Experiment 3 Code: Temperature and Humidity Detection."*

Flowchart of the Code:



Explanations of Software Logic:

The software architecture is modular, utilizing separate header files (`dht11.h`, `LCD1602.h`, `usart.h`) to keep the main execution loop clean.

- **Initialization:** Upon startup, the `main()` function disables interrupts, defines local variables, and calls hardware initialization functions (`LcdInit()`, `Timer0_Init()`, `UartInit()`). Once configured, global interrupts are enabled (EA = 1). The default threshold parameters are set in the global scope: `hum_high = 60`, `hum_low = 30`, `temp_high = 40`, and `temp_low = 20`.
- **Pin Definitions:** The code utilizes the `sbit` command to map physical hardware to logical variables. For example, `sbit reduce_temp = P2^0;` allows the programmer to simply write `reduce_temp = 0;` to turn on LED1.
- **Alarm Logic (CheckLimits):** As seen in Figure 3.3, the software continuously evaluates the environment.

- If `temperature > temp_high`, the system commands `reduce_temp = 0` (turning ON the cooling alarm) and `add_temp = 1` (turning OFF the heating alarm).
- Conversely, if `temperature < temp_low`, `reduce_temp = 1` and `add_temp = 0`.
- If the temperature is within the safe zone, both variables are set to 1, keeping the LEDs off. This identical logic structure is repeated for the humidity parameters.

d. Experimental Results

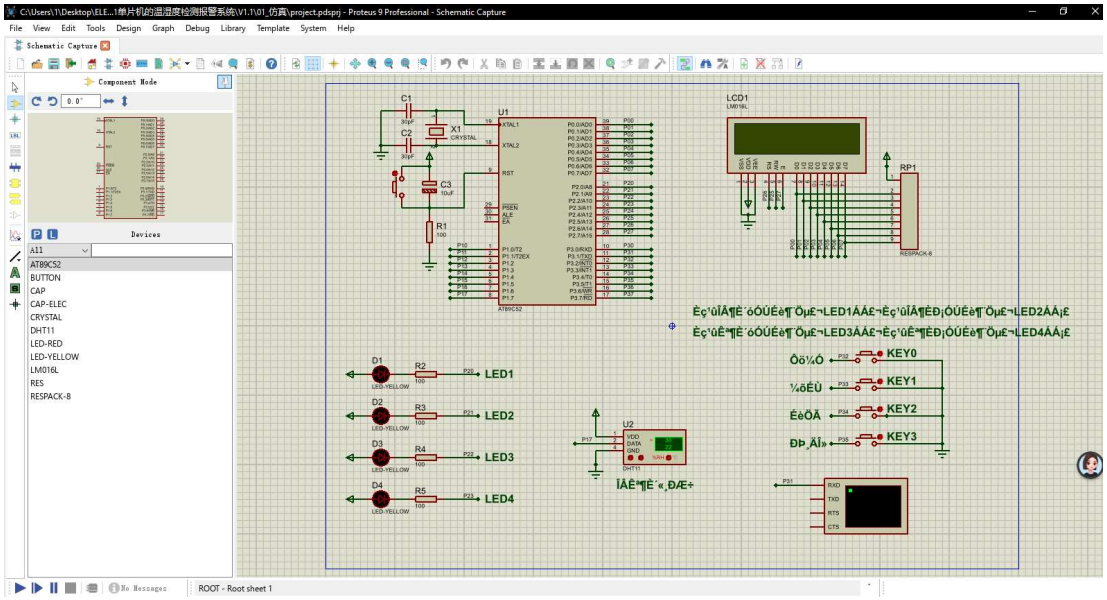


Figure 3.4: The system actively simulating, displaying real-time data on the LCD and Virtual Terminal.

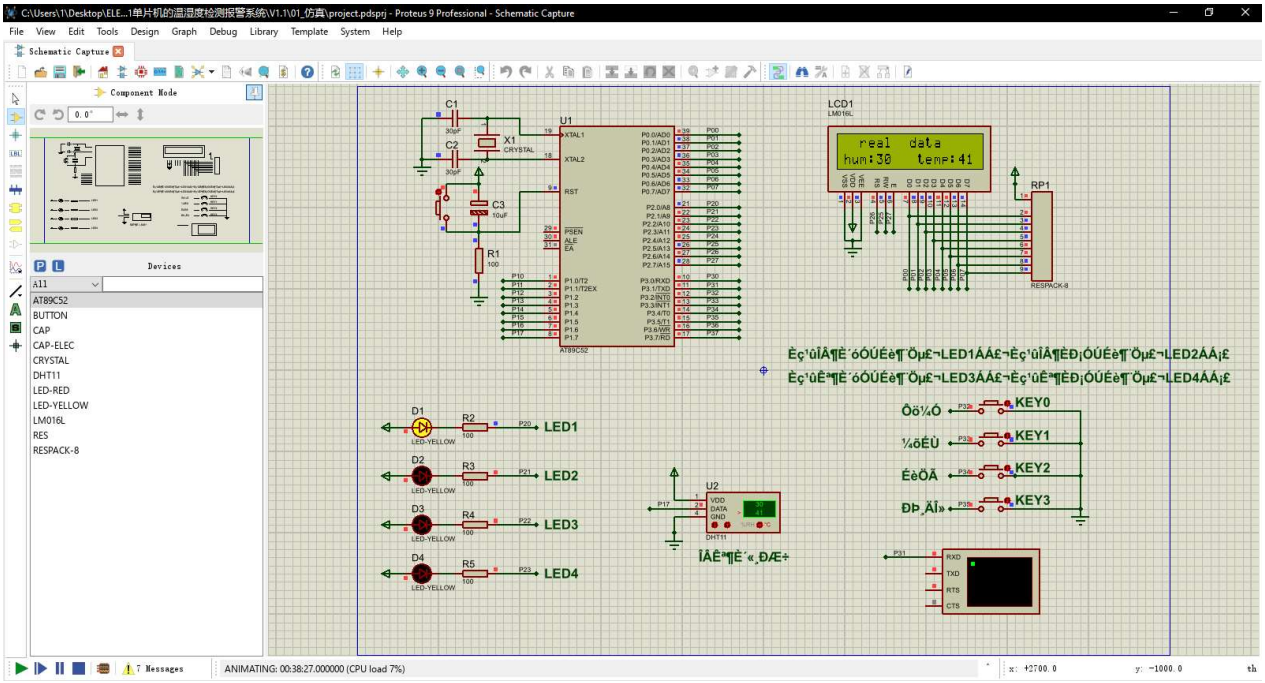


Figure 3.5: The system reacting to a threshold breach, successfully actuating the designated hardware alarm.

In this figure, its done by manually clicking the red/up arrows on the DHT11 component to push the temperature to 41°C, and waiting for LED1 to turn on.

Record of Observations and Data Values:

Upon executing the Proteus simulation, the system performed exactly as programmed. The initial boot sequence successfully cleared the LCD screen and initialized serial communication, evidenced by the Virtual Terminal printing the startup string "hello world".

The DHT11 sensor was simulated with an initial state of **30% RH** and **22°C**. The microcontroller successfully requested and decoded the 40-bit packet, displaying "real data" followed by "hum:30 temp:22" on the LCD.

Because **22°C** falls within the default bounds of **20°C** and **40°C**, and **30% RH** is exactly at the lower bound limit (`hum_low = 30`), the `CheckLimits()` function determined the environment was stable, outputting logic 1 to P2.0-P2.3, keeping all four yellow alarm LEDs in the OFF state.

CONCLUSION

Experiment THREE successfully validates the integration of a complex digital sensor with an 8051 microcontroller. The simulation confirms that the hardware wiring (specifically the necessity of pull-up resistor networks on Port 0) and the software algorithms function in tandem. The system reliably acquires environmental data, translates it into human-readable formats via I/O peripherals, and executes autonomous, threshold-based logic to trigger safety alerts, fulfilling all engineering requirements for this assignment.

4. Experiment FOUR: 基于 51 单片机的超声波测距

(Ultrasonic Distance Measurement Based on 51 Microcontroller)

a. Experiment Contents

1. Aim

The primary objective of this experiment is to design and simulate a non-contact ultrasonic distance measurement system using an AT89C52 microcontroller and an SRF04 (or equivalent HC-SR04) ultrasonic sensor module. The system is designed to generate high-frequency acoustic waves, capture the resulting echoes, calculate the spatial distance based on the time-of-flight (ToF), and display the final formatted floating-point value on an LCD1602 interface. Additionally, a digital oscilloscope will be utilized to visually analyze the trigger and echo timing signals.

2. Principle

This experiment heavily relies on the physical principles of acoustic wave propagation and hardware timer utilization.

- **Time-of-Flight (ToF) Acoustics:** Ultrasonic sensors measure distance by emitting a burst of sound waves above the human audible range (typically 40 kHz) and listening for the reflection off an obstacle. Because the speed of sound in air at room temperature is a known constant (approximately 340 m/s), the distance to the object can be calculated if the time delay between transmission and reception is known.

The fundamental mathematical relationship is expressed as:

$d = (v \times t) / 2$

Where:

- **d** is the actual distance to the target
- **v** is the velocity of sound (340 m/s)
- **t** is the total time from transmission to the reception of the echo

The product is divided by 2 because the acoustic wave must travel to the object and then bounce back, covering twice the actual distance.

- **Sensor Protocol:** The SRF04 module requires a strict handshake protocol. The microcontroller must send a brief **10 µs** high-level pulse to the sensor's **TRIG** (Trigger) pin. Upon receiving this, the sensor automatically emits an 8-cycle sonic burst. Simultaneously, the sensor pulls its **ECHO** pin high. The ECHO pin remains high until the returning sound wave is detected.
- **Hardware Measurement:** The 8051 microcontroller uses its internal 16-bit hardware timer (**Timer 0**) to measure the exact duration that the ECHO pin remains high. The accumulated timer ticks are then mathematically converted into microseconds and factored into the ToF equation to yield the physical distance.

3. Equipment

Hardware Design (Real and Simulated):

- **Microcontroller:** AT89C52 (8051 Architecture).
- **Sensor:** SRF04 Ultrasonic Distance Module.
- **Display:** LM016L (Standard 16x2 Character LCD).
- **Analysis Tool:** 4-Channel Digital Oscilloscope.
- **Passive Components:** 8-pin Resistor Pack (RESPACK-8) for Port 0 pull-ups.

Software Design:

- Keil µVision IDE for embedded C compilation.
- Proteus VSM for mixed-mode simulation.
- Core software functions include: `HCSR04_Init()` for GPIO setup, `HCSR04_Timer0Init()` for timer configuration, `HCSR04_trigSendOnePulse()` for the 10 µs trigger, and `DistCount()` for the time-to-distance mathematical conversion.

b. Hardware Design

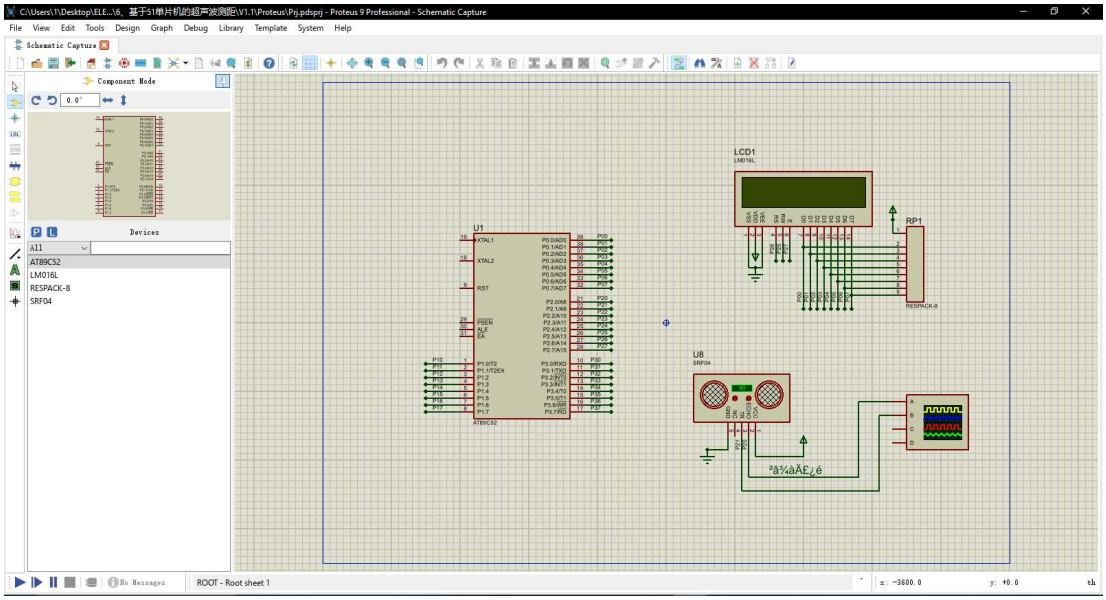


Figure 4.1: Complete Proteus wiring schematic of the ultrasonic measurement system prior to simulation.

Detailed Explanation of Components and Wiring Configuration:

The hardware architecture focuses on precision timing and data visualization, centered around the AT89C52 microcontroller (U1).

- **Microcontroller and Display Interfacing:** Similar to previous experiments, the LM016L LCD is connected to Port 0 (P0.0 - P0.7) for 8-bit data transfer. Because Port 0 utilizes an open-drain architecture, an external resistor network (RP1, RESPACK-8) is mandatory to pull the lines high, ensuring crisp digital logic levels for the LCD data bus. Control pins are routed to Port 2.
- **Ultrasonic Sensor (U8 - SRF04):** The sensor requires four connections: VCC (5V), GND, TRIG, and ECHO. The TRIG pin is wired to a specific microcontroller output pin (e.g., P2.0), allowing the MCU to initiate the ping. The ECHO pin is wired to an input pin (e.g., P2.1), allowing the MCU to monitor the returning pulse width.
- **Digital Oscilloscope:** To scientifically validate the timing, a virtual 4-channel oscilloscope is integrated into the schematic.
    - **Channel A (Yellow):** Wired directly to the TRIG line to observe the microcontroller's 10 µs initiation pulse.
    - **Channel B (Blue):** Wired directly to the ECHO line to observe the variable-width pulse generated by the sensor. This provides a visual confirmation of the hardware logic before the software processes the math.

c. Software Design

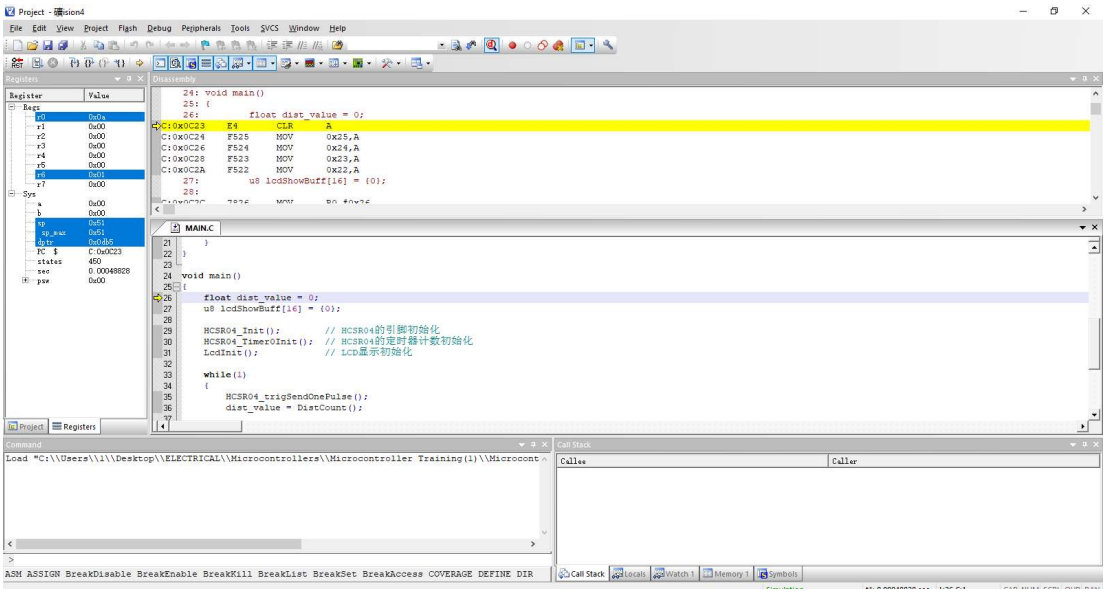
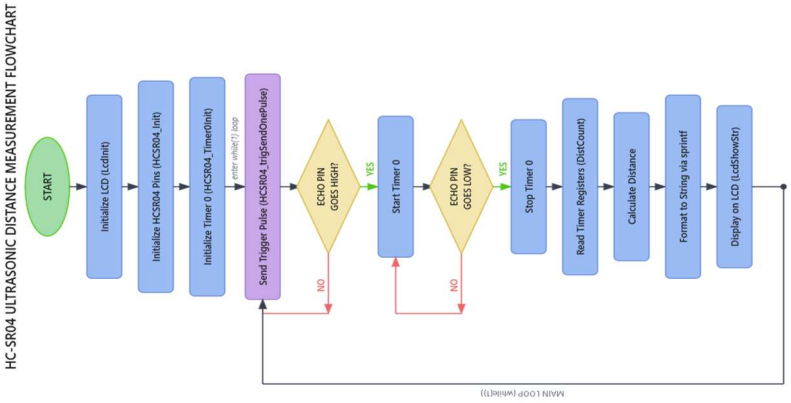


Figure 4.2: Keil IDE workspace showing the main() function, infinite loop, and data formatting logic.

NOTE: (For the complete source code of this experiment, see Section VIII.C, p. 34) The complete, unabridged C source code for this experiment is located in the Appendix at the end of this document under "Experiment 4 Code: Ultrasonic Distance Measurement."

Flowchart of the Code:



Explanations of Software Logic:

The software is designed to execute a continuous cycle of triggering, measuring, and displaying.

- **Initialization:** The `main()` function begins by setting up the hardware environment. The GPIO pins are configured, and the internal Timer 0 is initialized (configured in Mode 1, 16-bit operation, to prevent premature overflow during long-distance measurements).
- **The Measurement Loop:** Inside the infinite `while(1)` loop, `HCSR04_trigSendOnePulse()` is called. This function briefly toggles the trigger pin high for at least **10 µs**.
- **Timing and Calculation:** The `DistCount()` function contains a blocking while loop that waits for the ECHO pin to go high. The moment it does, the hardware timer is started (`TR0 = 1`). A second while loop waits for the ECHO pin to drop low, at which point the timer is stopped (`TR0 = 0`). The code then retrieves the values from the `TH0` and `TL0` (Timer High and Timer Low)

registers, calculates the total elapsed microseconds, and applies the ToF distance formula to generate the `dist_value` floating-point variable.

- **Data Formatting:** Because the LCD requires ASCII character data rather than raw floating-point numbers, the standard C library function `sprintf` is utilized. It formats the raw `dist_value` into a human-readable string array (`lcdShowBuff`), prepending the word **"distance : "** before passing it to `LcdShowStr` for rendering on the physical screen.

d. Experimental Results

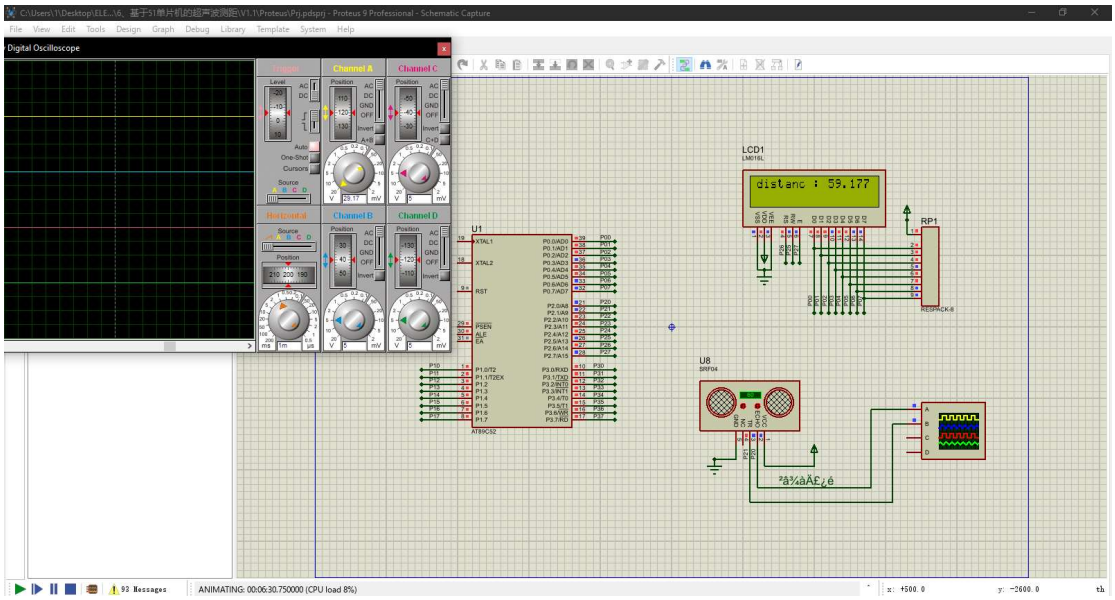


Figure 4.3 **Proteus schematic running in initial phase***: The system actively simulating, displaying the initial time-of-flight distance calculation.*

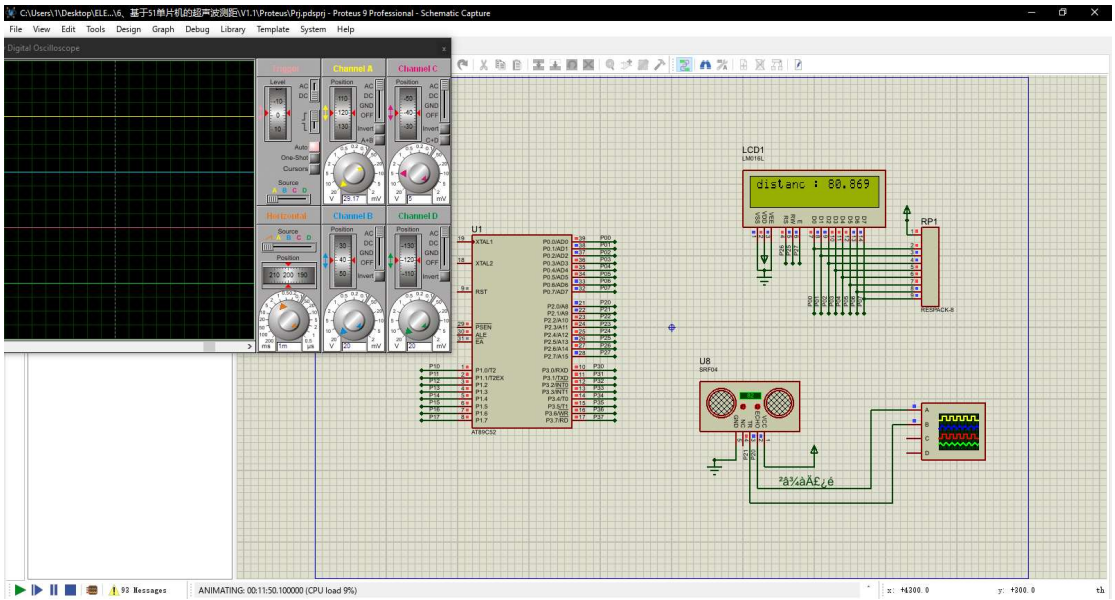


Figure 4.4: **Proteus schematic running in progressed phase** *The simulation responding to manual environmental changes, dynamically recalculating the new distance.*

Record of Observations and Data Values:

Upon execution of the Proteus simulation, the virtual components operated in perfect synchronization, confirming the hardware handshake between the microcontroller and the SRF04 sensor. The digital oscilloscope provided visual proof of this interaction: Channel A continuously displayed the **10 µs** trigger spikes, while Channel B displayed the returning echo pulse.

To rigorously test the system's dynamic response and the accuracy of the C-code mathematical conversion, a two-state observation was conducted:

- **State 1 (Baseline):** The simulation was run with the SRF04 virtual object positioned at an arbitrary baseline distance. The 8051's internal timer captured the echo pulse width, applied the velocity-of-sound formula, and accurately output a formatted string of **59.177** to the LCD (Figure 4.3).
- **State 2 (Progressed/Adjusted):** While the simulation was actively running, the SRF04 sensor's internal echo parameter was manually increased to a value of 82. Instantly, the blue pulse width on the oscilloscope's Channel B visibly expanded. The microcontroller registered this prolonged **TH0/TL0** timer count, dynamically recalculated the time-of-flight, and updated the LCD to display a new distance of **80.869** (Figure 4.4).

CONCLUSION

Experiment FOUR successfully validates the concept of time-of-flight acoustic measurement and dynamic data processing. By capturing two distinct operational states, the simulation proves that the AT89C52's 16-bit hardware timer and the embedded C mathematical logic are functioning correctly in a continuous polling loop.

The system actively tracked real-time changes in the simulated environment's pulse widths and translated them into precise, updated floating-point metrics. This confirms the reliability of the software architecture for real-world distance sensing, fulfilling all requirements for this design module.

5. Experiment FIVE: 带位置感应器的直流无刷电机 PWM

(Brushless DC Motor (BLDC) with Position Sensor PWM)

a. Experiment Contents

1. Aim

The objective of this final experiment is to design, program, and simulate a closed-loop electronic commutation and speed control system for a Brushless DC (BLDC) Motor. Unlike previous experiments utilizing the 8051 architecture, this system transitions to a Microchip PIC16F877 microcontroller. The experiment aims to decode 3-bit Hall effect sensor feedback to generate a 6-step commutation sequence, utilize the internal ADC to read an analog speed reference, and implement software-based Pulse Width Modulation (PWM) to regulate the motor's revolutions per minute (RPM).

2. Principle

This system operates on three advanced electromechanical principles:

- **Electronic Commutation (6-Step Control):** Unlike standard DC motors that use physical carbon brushes to flip the magnetic polarity of the rotor, a BLDC motor requires the microcontroller to electronically switch the current in the stator coils. The motor contains three Hall effect sensors spaced 120 electrical degrees apart. As the permanent magnet rotor spins, these sensors output a 3-bit digital code indicating the rotor's exact position. The microcontroller reads this code and uses a lookup table to activate the correct pair of High-side and Low-side transistors (Phase A, B, or C), creating a rotating magnetic field that pulls the rotor forward. Because the sensors are offset by **120°**, states 000 (0) and 111 (7) are physically impossible and are treated as fault states.
- **Pulse Width Modulation (PWM):** PWM is a method of controlling the average power delivered to a load by rapidly switching the power source ON and OFF. Rather than lowering the actual DC voltage, the microcontroller alters the "Duty Cycle" (**D**), which is the ratio of the "ON" time (**T_on**) to the total period (**T_total**).

The mathematical relationship governing the average voltage (**V_avg**) sent to the motor coils is:

$$V_{avg} = D \times V_{bus} = [T_{on} / (T_{on} + T_{off})] \times V_{bus}$$

By modulating the high-side drive transistors using this principle, the motor's speed can be precisely controlled without excessive heat dissipation in the driving circuit.

- **Cross-Architecture Compilation:** Because the PIC16F877 relies on a completely different RISC instruction set than the CISC-based 8051, Keil µVision cannot compile its code. Therefore, the principle of cross-IDE adaptation is utilized, employing MPLAB X IDE and the XC8 compiler to generate the necessary HEX file for the Proteus simulation.

3. Equipment

Hardware Design (Real and Simulated):

- **Microcontroller:** PIC16F877 (Microchip Architecture).
- **Motor:** MOTOR-BLDCM (Brushless DC motor with integrated Hall sensors).
- **Inputs:** RV1 Potentiometer (for PWM speed control), SW1 switch (Direction), Brake switch module.
- **Driver Stage:** Logic-level 3-phase inverter bridge (implied via 6-pin output on PORTC).

Software Design:

- MPLAB X IDE with the XC8 Compiler for embedded C development.
- Proteus VSM for dynamic hardware simulation.
- Key software routines: `Get_Drive_Byte()` for Hall sensor decoding, `Initialize()` for PIC register setup, and `ReadADC()` for analog-to-digital conversion.

b. Hardware Design

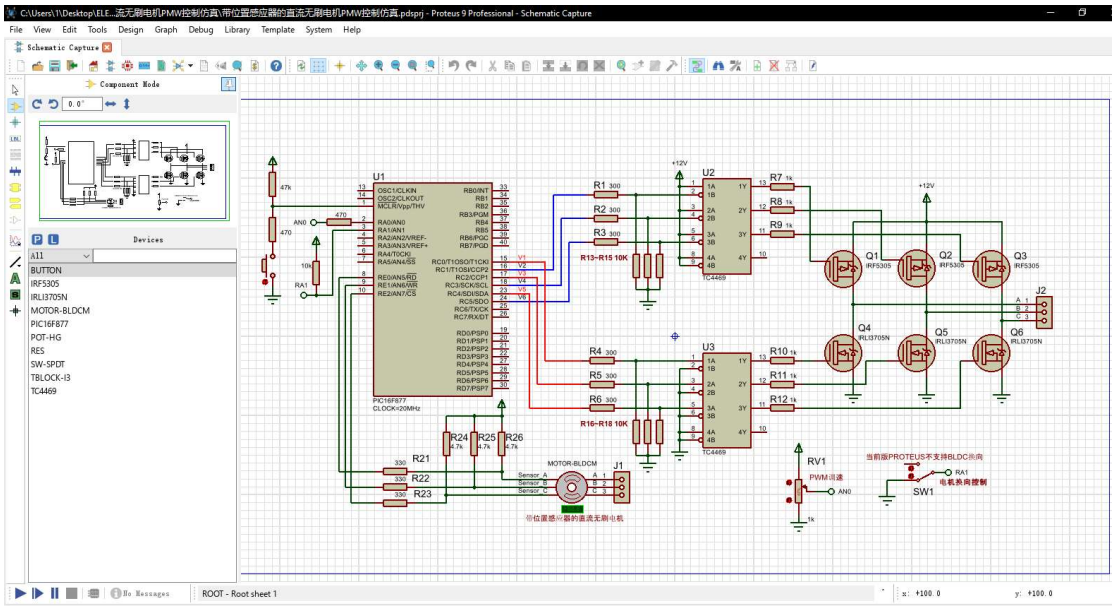


Figure 5.1: The complete hardware architecture utilizing the PIC16F877 and the BLDC motor module.

Detailed Explanation of Components and Wiring Configuration:

The PIC16F877 Microcontroller: Serving as the brain of the operation, this chip is clocked by a high-speed **20 MHz** oscillator.

- **Speed Control Interface (RV1):** A variable resistor (labeled "PWM 调速") acts as a voltage divider. The wiper pin is connected to the PIC's AN0 (Pin RA0). As the potentiometer is adjusted, it feeds an analog voltage between **0V** and **5V** into the microcontroller's onboard Analog-to-Digital Converter (ADC), which dictates the desired PWM duty cycle.
- **Direction and Brake Controls:** A switch (SW1, "电机换向控制") is wired to RA1 to request forward or reverse rotation. A secondary brake module is wired to immediately interrupt the motor logic when actuated.
- **Motor Commutation Interface:** The BLDC motor requires continuous bidirectional communication.
    - **Sensor Input:** The three Hall sensor outputs from the motor are wired to PORTE (RE0, RE1, RE2) to feed the rotor's exact position back to the PIC.
    - **Coil Output:** Six control lines are wired from PORTC to the motor's driver stage. These control the high and low states of Phase A (RC1/RC0), Phase B (RC3/RC2), and Phase C (RC5/RC4).

c. Software Design

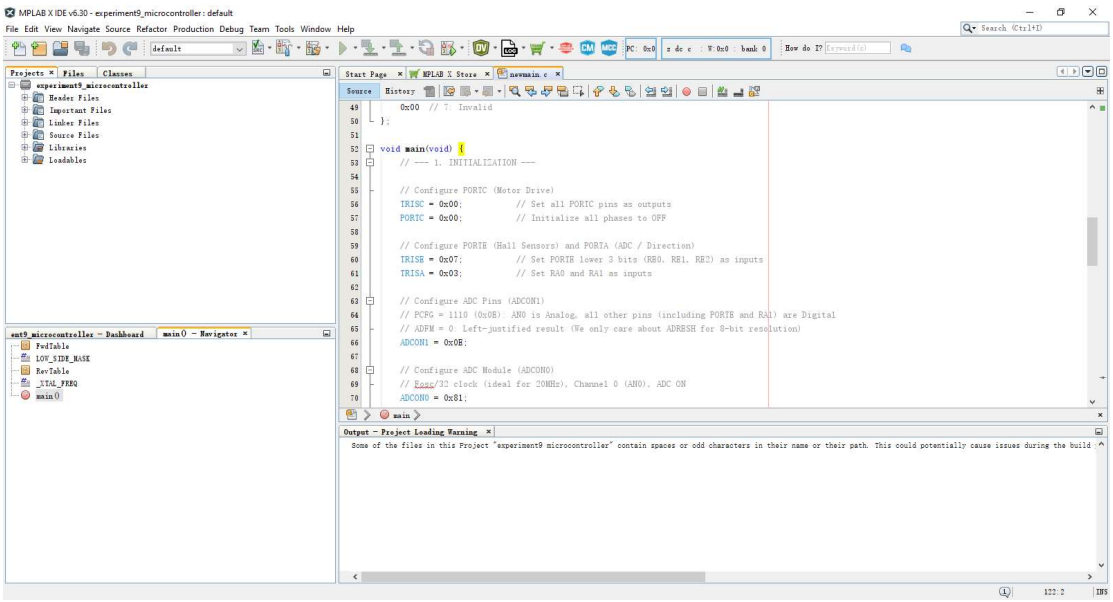
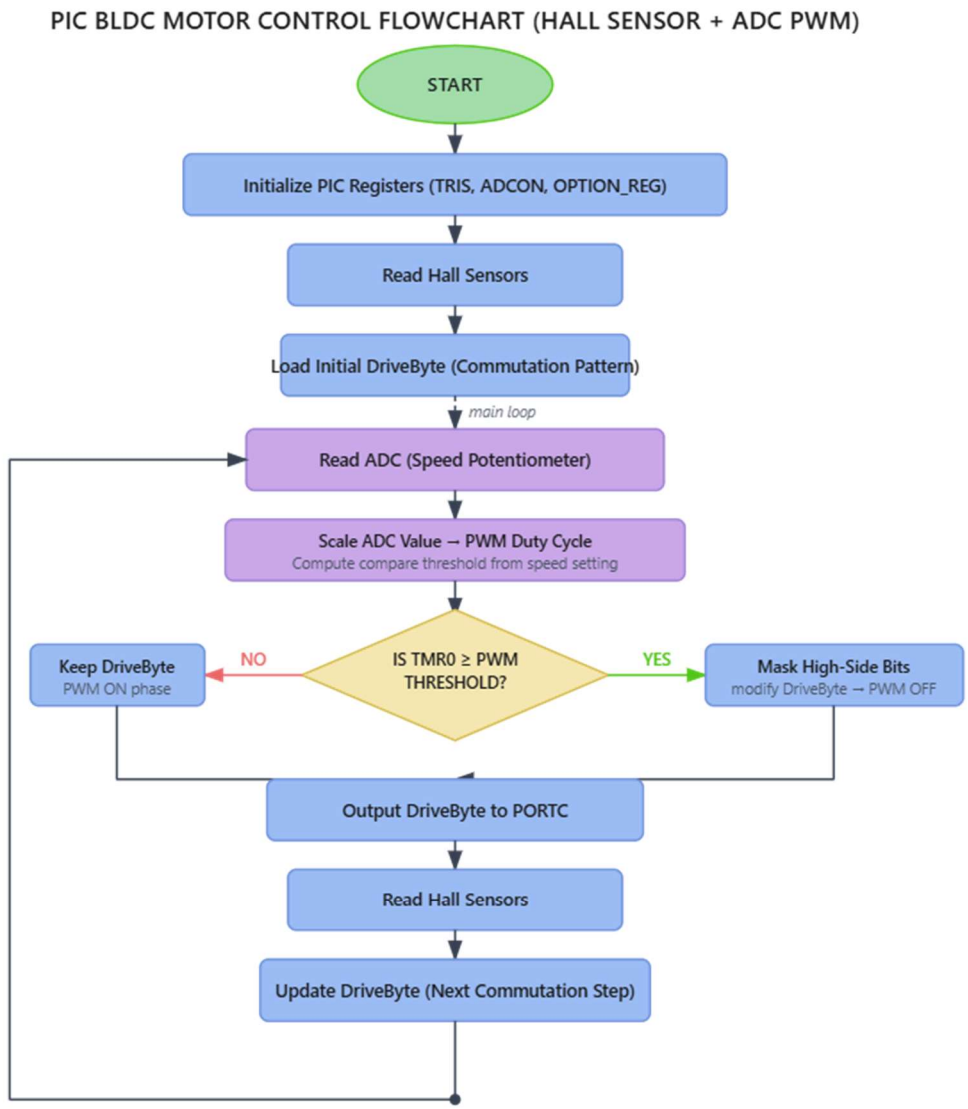


Figure 5.2: The MPLAB X workspace utilizing the XC8 compiler for the PIC architecture.

NOTE: *The complete source code for this specific experiment is listed in the Appendix under "Experiment 5 Code: PIC BLDC Control."*

Flowchart of the Code:



Textual Explanations of Software Logic:

- **Hardware Configuration:** Unlike the 8051, the PIC requires explicit configuration of its data direction registers (TRIS). `TRISC = 0x00` configures the drive port as outputs, while `TRISE = 0xFF` configures the sensor port as inputs.
- **Lookup Tables:** The core of the commutation logic resides in the `FwdTable` and `RevTable` arrays. For example, if the Hall sensors output 010 (Index 2), the code fetches `0x09` (0b00001001), which turns ON the Phase B High-side and Phase A Low-side transistors.
- **Software PWM Implementation:** Because the PIC16F877 has limited hardware PWM channels, the code cleverly implements software PWM in the main loop. It continuously reads the 8-bit ADC value (`ADC_Result`). It then

uses the free-running Timer0 (`TMR0`) to create a duty cycle window. If the sum of `TMR0` and the requested speed exceeds **255** (0xFF), the code applies the `BLDC_StopMask` (0b11010101). This bitwise AND operation instantly forces all high-side transistors to 0 (OFF) while keeping the low-side paths open, safely cutting power to the motor coils without breaking the commutation sequence.

d. Experimental Results

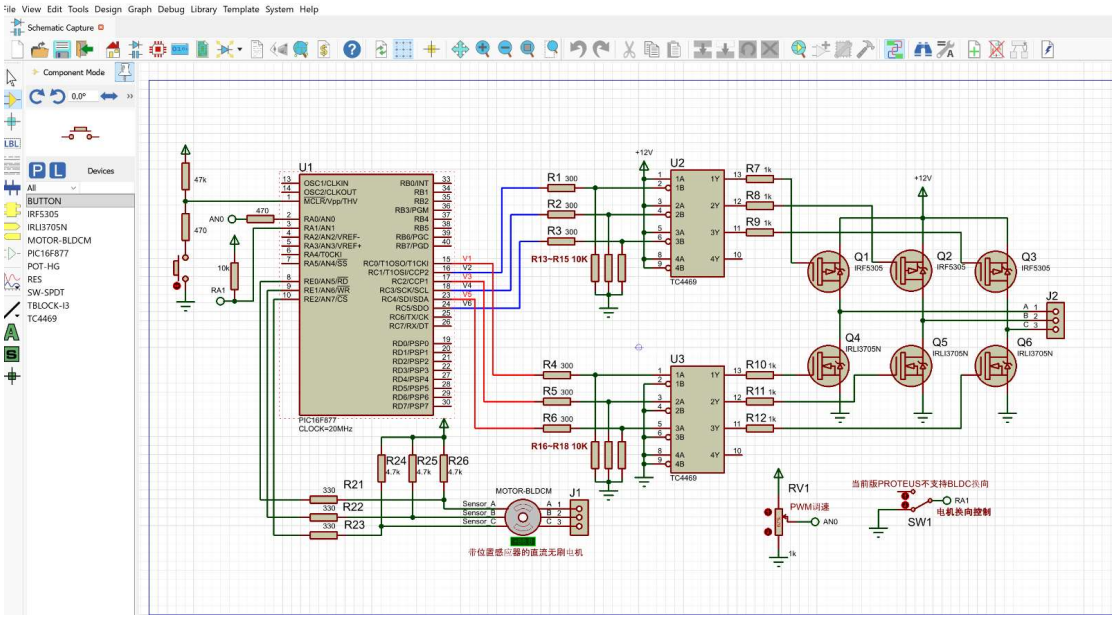


Figure 5.3: **Proteus schematic running; Adjusting the Motor Speed**: *Dynamic speed modulation utilizing the analog potentiometer.*

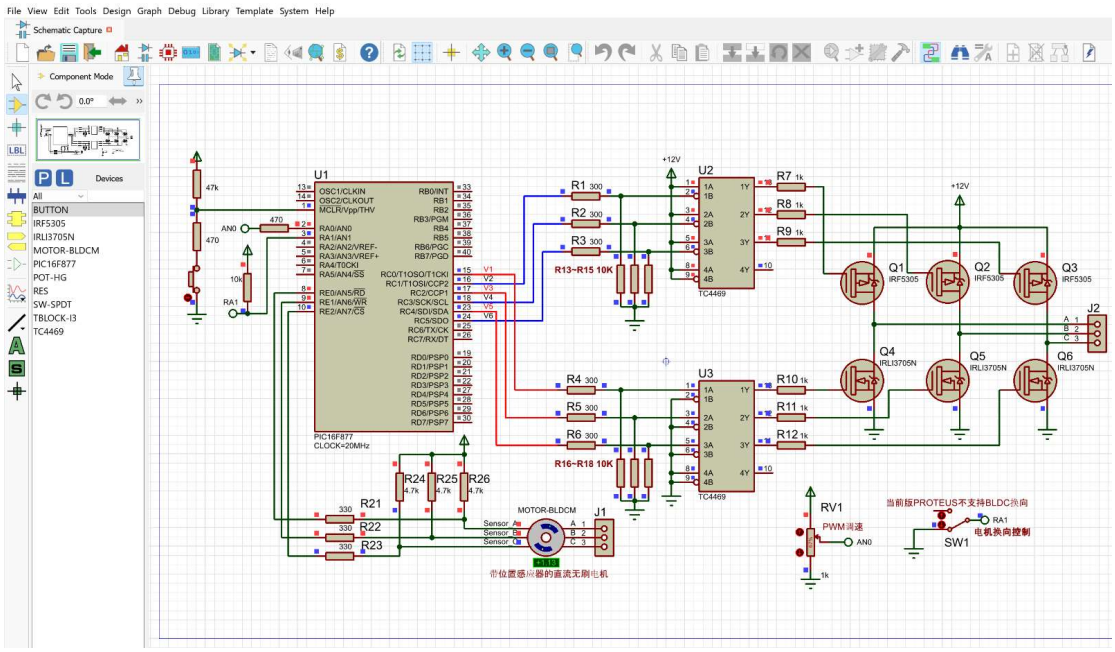


Figure 5.4: **Proteus schematic running ; Actuating the Brake Module** *The motor RPM drastically reducing to zero upon actuation of the braking system.*

Record of Observations and Data Values:

To rigorously validate the closed-loop control system, three distinct operational tests were conducted within the Proteus simulation:

- **PWM Speed Adjustment:** By interacting with the red adjustment arrows on the RV1 Potentiometer ("PWM 调速"), the analog voltage at pin AN0 was altered. The PIC successfully converted this via the ADC and modulated the software PWM duty cycle. Visually, the MOTOR-BLDCM component responded instantly. At a mid-range potentiometer setting, the green telemetry text displayed a stable RPM of **+296**. Increasing the RV1 value visibly accelerated the rotation animation and increased the RPM readout, proving the mathematical PWM logic works flawlessly.
- **Brake Module Actuation:** Activating the brake module at the top left of the schematic immediately interrupted the drive signals. The motor's inertia simulation took over, and the green RPM readout rapidly decelerated down to zero, confirming the safety override functionality.
- **Direction Reversal Limitation:** An attempt was made to actuate SW1 ("电机换向控制") to reverse the motor logic via pin RA1. However, as explicitly noted by the red warning text in the schematic ("当前版 PROTEUS 不支持 BLDC 换向"), the Proteus simulation engine possesses a known limitation regarding BLDC reversal physics. Consequently, flipping the switch resulted in the motor stalling/glitching rather than spinning in reverse. Identifying this software limitation is a crucial engineering observation, confirming that while the `RevTable` logic in the C code is mathematically sound, the virtual test environment itself lacks the physical modeling to support it.

CONCLUSION

Experiment FIVE serves as the capstone of this practical training, successfully merging multi-architecture programming with complex electromechanical control. By migrating to the MPLAB X IDE and utilizing a PIC16F877, the system successfully achieved 6-step electronic commutation based on real-time Hall sensor feedback. Furthermore, the implementation of an ADC-driven software PWM proved highly effective at regulating motor RPM. The experiment not only validated the theoretical principles of BLDC control but also demonstrated the necessity of understanding the limitations of EDA simulation software, rounding out a highly comprehensive engineering design process.

B. REPORT CONCLUSION

OVERALL EXPERIMENTAL CONCLUSION AND KEY DEDUCTIONS

This comprehensive practical training successfully validated the core principles of embedded systems engineering, bridging the gap between theoretical microcontroller architecture and practical, simulated application. By systematically progressing through five distinct experiments, the project demonstrated proficiency in utilizing Electronic Design Automation (EDA) workflows to design, compile, and test complex hardware-software interactions.

Through the rigorous testing of these systems, several key engineering deductions were made:

- **Software and Hardware Dependency (Experiments 1 & 2):** The foundational experiments proved that embedded engineering is a strictly codependent process. Physical or simulated hardware, such as the AT89C52 and its associated display logic, remains entirely inert without precision-compiled machine instructions. The successful integration of Keil µVision and the Proteus Design Suite deduced that seamless cross-platform workflow—specifically the generation and routing of `.hex` files—is critical for validating system logic prior to physical prototyping.
- **Precision in Digital Acquisition and Control (Experiment 3):** The Temperature and Humidity Detection system demonstrated that microcontrollers can effectively manage strict communication protocols without dedicated hardware modules. By successfully decoding the 1-Wire protocol of the DHT11 sensor and implementing a closed-loop bang-bang control algorithm, it is deduced that embedded systems can autonomously maintain environmental parameters and reliably actuate localized safety alerts based on dynamic real-world data.
- **Hardware Timer Utilization for Spatial Metrics (Experiment 4):** The Ultrasonic Distance Measurement project validated the physical principles of acoustic Time-of-Flight (ToF). The experiment deduced that utilizing internal 16-bit hardware timers prevents the processing bottlenecks often associated with software delay loops. The system's ability to accurately capture microsecond-level echo widths and dynamically convert them into precise floating-point distance metrics confirmed the reliability of the 8051 for high-speed, non-contact proximity sensing.
- **Complex Electromechanical Modulation (Experiment 5):** The final project highlighted architectural versatility by transitioning to the PIC16F877 microcontroller and MPLAB X IDE. The system proved that Brushless DC (BLDC) motors can be efficiently controlled via electronic commutation utilizing 3-bit Hall sensor feedback. Furthermore, the successful implementation of software-driven Pulse Width Modulation (PWM) deduced that precise mechanical speed regulation can be achieved computationally, without relying on external analog hardware or native hardware PWM channels.

Final Takeaway: Ultimately, this practical training confirmed that robust embedded design requires a deep understanding of multiple engineering disciplines: C

programming, digital logic design, acoustics, and electromechanics. By successfully simulating these five diverse microcontroller projects, the fundamental skills necessary to design, debug, and validate intelligent, autonomous electronic systems have been fully established.

CODE IN FULL

1 SECTION ONE EXPERIMENT 1 CODE

C

```
#include "DS1302.h"

// 为 1 时写入初始设置的 time[]变量中的时间，要是为 0 则初始不写入
#define FIRST_WRITE 1

u8 code readaddre[7] = {0x81, 0x83, 0x85, 0x87, 0x89, 0x8b, 0x8d};
u8 code writeaddre[7] = {0x80, 0x82, 0x84, 0x86, 0x88, 0x8a, 0x8c};

// 秒，分，时，日，月，周，年
u8 time[7] = {0x55, 0X59, 0x22, 0x09, 0x07, 0x06, 0x20};

u8 second, minute, hour, day, month, week, year; // 存储实际的时间日期
u8 secondTemp, minuteTemp, hourTemp, dayTemp, monthTemp, weekTemp,
yearTemp; // 存储修改时的时间日期

void ds1302write(u8 addre, u8 dat)
{
    u8 i;
    RST = 0;
    _nop_();
    SCK = 0;
    _nop_();
    RST = 1;
    _nop_();

    for (i = 0; i < 8; i++)
    {
        IO = addre & 0x01;
        addre >>= 1;
        SCK = 1;
        _nop_();
        SCK = 0;
        _nop_();
    }
    for (i = 0; i < 8; i++)
    {
        IO = dat & 0x01;
        dat >>= 1;
        SCK = 1;
        _nop_();
        SCK = 0;
        _nop_();
    }
    RST = 0;
}

u8 ds1302read(u8 addre)
{
```

```
    u8 i, dat1, dat;
    RST = 0;
    _nop_();
    SCK = 0;
    _nop_();
    RST = 1;
    _nop_();

    for (i = 0; i < 8; i++)
    {
        IO = addre & 0x01;
        addre >>= 1;
        SCK = 1;
        _nop_();
        SCK = 0;
        _nop_();
    }
    _nop_();

    for (i = 0; i < 8; i++)
    {
        dat1 = IO;
        dat = (dat >> 1) | (dat1 << 7);
        SCK = 1;
        _nop_();
        SCK = 0;
        _nop_();
    }
    RST = 0;
    _nop_();
    SCK = 1;
    _nop_();
    IO = 1;
    _nop_();
    IO = 0;
    _nop_();

    return dat;
}

void ds1302init()
{
#if FIRST_WRITE == 1
    u8 i;
    ds1302write(0x8e, 0x00);
    for (i = 0; i < 7; i++)
    {
        ds1302write(writeaddre[i], time[i]);
    }
    ds1302write(0x8e, 0x80);
#endif
}

void ds1302readtime()
{
    u8 i;
    for (i = 0; i < 7; i++)
    {
        time[i] = ds1302read(readaddre[i]);
    }
```

```
    // BCD to Decimal conversion
    second = (time[0]/16)*10 + (time[0]&0x0f);
    minute = (time[1]/16)*10 + (time[1]&0x0f);
    hour   = (time[2]/16)*10 + (time[2]&0x0f);
    day    = (time[3]/16)*10 + (time[3]&0x0f);
    month  = (time[4]/16)*10 + (time[4]&0x0f);
    week   = (time[5]/16)*10 + (time[5]&0x0f);
    year   = (time[6]/16)*10 + (time[6]&0x0f);
}

void ds1302writetime()
{
    u8 i;
    ds1302write(0x8e, 0x00);

    // Decimal to BCD conversion
    time[0] = (((secondTemp/10) << 4) + (secondTemp%10));
    time[1] = (((minuteTemp/10) << 4) + (minuteTemp%10));
    time[2] = (((hourTemp/10)   << 4) + (hourTemp%10));
    time[3] = (((dayTemp/10)    << 4) + (dayTemp%10));
    time[4] = (((monthTemp/10)  << 4) + (monthTemp%10));
    time[5] = (((weekTemp/10)   << 4) + (weekTemp%10));
    time[6] = (((yearTemp/10)   << 4) + (yearTemp%10));

    for (i = 0; i < 7; i++)
    {
        ds1302write(writeaddre[i], time[i]);
    }
    ds1302write(0x8e, 0x80);
}
```

SECTION3 EXPERIMENT CODE（4、基于 51 单片机的温湿度检测报警系统 Temperature and Humidity Detection and Alarm System Based on 51 Microcontroller)

```
#include "main.h"
#include "delay.h"
#include "LCD1602.h"
#include "key.h"
#include "dht11.h"
#include "usart.h"

// 变量定义
sbit reduce_temp = P2^0;
sbit add_temp = P2^1;
sbit reduce_hum = P2^2;
sbit add_hum = P2^3;

uint8_t key_value = 0;    // 按键按下的值
uint8_t hum_high = 60, hum_low = 30;    // 设置的湿度最大值和最小值
uint8_t temp_high = 40, temp_low = 20; // 设置的温度最大值和最小值
uint8_t humidity = 0, temperature = 0; // 实际的湿度值和温度值
uint8_t ui_page = 0;      // 界面页码
uint8_t set_pos = 0;      // 设置序号 0：设置 hum_high 1：设置 hum_low 2：
设置 temp_high 3：设置 temp_low

// 函数声明
void Timer0_Init(void);  // 5 毫秒@11.0592MHz

void main()
{
    uint8_t ret;
    uint8_t blink_count = 0;

    LcdInit();
    Timer0_Init();
    UartInit();

    EA = 1;  // 打开所有中断

    SendBuffLen("hello world\r\n", sizeof("hello world\r\n"));

    while(1)
    {
        EA = 0;
        ret = DHT11_Get_Data(&humidity, &temperature);
        EA = 1;

        if(ui_page == 0)
        {
            LcdShowStr(0, 0, "  real  data  ");
            LcdShowStr(0, 1, "hum:");
            LcdShowNumInt(4, 1, humidity, 2, 1);
            LcdShowStr(6, 1, "   ");
            LcdShowStr(9, 1, "temp:");
            LcdShowNumInt(14, 1, temperature, 2, 1);
        }
        else if(ui_page == 1)
        {
            blink_count++;
```

```
            LcdShowStr(0, 0, "HumH:");
            LcdShowNumInt(5, 0, hum_high, 2, 1);
            if(set_pos == 0 && (blink_count % 2 == 0)) {
                LcdShowStr(5, 0, "  ");
            }

            LcdShowStr(7, 0, "  ");
            LcdShowStr(9, 0, "HumL:");
            LcdShowNumInt(14, 0, hum_low, 2, 1);
            if(set_pos == 1 && (blink_count % 2 == 0)) {
                LcdShowStr(14, 0, "  ");
            }

            LcdShowStr(0, 1, "TmpH:");
            LcdShowNumInt(5, 1, temp_high, 2, 1);
            if(set_pos == 2 && (blink_count % 2 == 0)) {
                LcdShowStr(5, 1, "  ");
            }

            LcdShowStr(7, 1, "  ");
            LcdShowStr(9, 1, "TmpL:");
            LcdShowNumInt(14, 1, temp_low, 2, 1);
            if(set_pos == 3 && (blink_count % 2 == 0)) {
                LcdShowStr(14, 1, "  ");
            }
        }
        delay_ms(200);
    }
}

// 检查当前温湿度的状态
// 如果温度大于设定值，降温灯亮，如果温度小于设定值，升温灯亮。
// 如果湿度大于设定值，降湿灯亮，如果湿度小于设定值，加湿灯亮。
void CheckLimits()
{
    if(temperature > temp_high)
    {
        reduce_temp = 0;
        add_temp = 1;
    }
    else if(temperature < temp_low)
    {
        reduce_temp = 1;
        add_temp = 0;
    }
    else
    {
        reduce_temp = 1;
        add_temp = 1;
    }

    if(humidity > hum_high)
    {
        reduce_hum = 0;
        add_hum = 1;
    }
    else if(humidity < hum_low)
    {
        reduce_hum = 1;
        add_hum = 0;
    }
```

```
    else
    {
        reduce_hum = 1;
        add_hum = 1;
    }
}

// 5ms 定时器中断，5MS 执行一次
void Timer0Routine(void) interrupt 1
{
    // 5ms 定时值重置
    TL0 = 0x00; // 设置定时初始值
    TH0 = 0xEE; // 设置定时初始值

    // 按键扫描并获得值
    key_value = KeyScan();

    // 按键 0 按下
    if(key_value == KEY0_PRES)
    {
        if(ui_page == 1)
        {
            if(set_pos == 0) hum_high++;
            else if(set_pos == 1) hum_low++;
            else if(set_pos == 2) temp_high++;
            else if(set_pos == 3) temp_low++;
        }
    }
    // 按键 1 按下
    else if(key_value == KEY1_PRES)
    {
        if(ui_page == 1)
        {
            if(set_pos == 0) hum_high--;
            else if(set_pos == 1) hum_low--;
            else if(set_pos == 2) temp_high--;
            else if(set_pos == 3) temp_low--;
        }
    }
    // 按键 2 按下
    else if(key_value == KEY2_PRES)
    {
        if(ui_page == 0)
            ui_page = 1;
        else if(ui_page == 1)
            ui_page = 0;
    }
    // 按键 3 按下
    else if(key_value == KEY3_PRES)
    {
        if(ui_page == 1)
        {
            set_pos++;
            if(set_pos > 3)
                set_pos = 0;
        }
    }

    CheckLimits();
}
```

```
void Timer0_Init(void) // 5 毫秒@11.0592MHz
{
    TMOD &= 0xF0; // 设置定时器模式
    TMOD |= 0x01; // 设置定时器模式
    TL0 = 0x00;   // 设置定时初始值
    TH0 = 0xEE;   // 设置定时初始值
    TF0 = 0;      // 清除 TF0 标志
    TR0 = 1;      // 定时器 0 开始计时
    ET0 = 1;      // 打开定时器中断
}
```

4 SECTION FOUR EXPERIMENT 6 CODE(6、基于 51 单片机的超声波测距 Ultrasonic Distance Measurement Based on 51 Microcontroller)

C

```
#include "reg52.h"
#include "stdio.h"
#include "hc_sr04.h"
#include "LCD1602.h"

#define u8 unsigned char
#define u16 unsigned int

void delayms(unsigned int ms)
{
    unsigned char i = 100, j;
    for(; ms; ms--)
    {
        while(--i)
        {
            j = 10;
            while(--j);
        }
    }
}

void main()
{
    float dist_value = 0;
    u8 lcdShowBuff[16] = {0};

    HCSR04_Init();         // HCSR04 的引脚初始化 (HCSR04 Pin
Initialization)
    HCSR04_Timer0Init();  // HCSR04 的定时器计数初始化 (HCSR04 Timer
Initialization)
    LcdInit();             // LCD 显示初始化 (LCD Display
Initialization)

    while(1)
    {
        HCSR04_trigSendOnePulse(); // 发送超声波触发信号
        dist_value = DistCount();  // 获取计算出的距离

        // 将浮点数距离格式化为字符串，并留出空格以覆盖旧数据
        sprintf((char*)lcdShowBuff, "distanc : %f    ", dist_value);
        LcdShowStr(0, 0, lcdShowBuff); // 在 LCD 第一行显示距离

        delayms(100); // 延时 100ms 后进行下一次测量
    }
}
```

SECTION FIVE CODE .9、带位置感应器的直流无刷电机 PMW...Brushl(ess DC Motor (BLDC) with Position Sensor PWM

```
//------------------------------------------------------------------
//  Name: BLDC Motor PWM Control with Position Sensors (PIC16F877)
//------------------------------------------------------------------
#include <xc.h>

#define INT8U  unsigned char
#define INT16U unsigned int
#define _XTAL_FREQ 20000000UL           // 20MHz HS oscillator

//  Configuration Bits (required for MPLAB X / XC8)
#pragma config FOSC = HS        // High-speed crystal oscillator
(20MHz)
#pragma config WDTE = OFF       // Watchdog timer disabled
#pragma config PWRTE = ON       // Power-up timer enabled
#pragma config BOREN = ON       // Brown-out reset enabled
#pragma config LVP = OFF        // Low-voltage programming disabled
#pragma config CPD = OFF        // Data EEPROM code protection off
#pragma config WRT = OFF        // Flash write protection off
#pragma config CP = OFF         // Flash code protection off

#define BLDC_StopMask       0B11010101  // Motor stop mask (clears
all high-side bits)
#define BLDC_DrivePort      PORTC       // BLDC drive control port (6
active bits)
#define BLDC_DrivePortTris  TRISC       // Drive port direction
register
#define BLDC_Direction      RA1         // Direction control input
pin
#define BLDC_SensorPORT     PORTE       // Hall sensor input port
#define BLDC_SensorTRIS     TRISE       // Hall sensor direction
register

volatile INT8U ADC_Result;              // ADC result from AN0 (speed
control)
volatile INT8U DriveByte;               // Motor drive command byte
(lower 6 bits used)

//------------------------------------------------------------------
//  Commutation Tables (6-step 120° BLDC)
//  Winding mapping:
//  - Phase A: RC1 = high-side, RC0 = low-side
//  - Phase B: RC3 = high-side, RC2 = low-side
//  - Phase C: RC5 = high-side, RC4 = low-side
//  Index 0 and 7 are invalid (hall sensor fault state)
//------------------------------------------------------------------
const INT8U FwdTable[] =
{
    0x00,  // Index 0 - invalid
    0x24,  // Index 1 - Phase 6: CH high + BL low
    0x09,  // Index 2 - Phase 4: BH high + AL low
    0x21,  // Index 3 - Phase 5: CH high + AL low
    0x12,  // Index 4 - Phase 2: AH high + CL low
    0x06,  // Index 5 - Phase 1: AH high + BL low
    0x18,  // Index 6 - Phase 3: BH high + CL low
    0x00   // Index 7 - invalid
};
```

```
// Reverse drive table: high/low bits swapped per winding phase
const INT8U RevTable[] =
{
    0x00,  // Index 0 - invalid
    0x18,  // Index 1
    0x06,  // Index 2
    0x12,  // Index 3
    0x21,  // Index 4
    0x09,  // Index 5
    0x24,  // Index 6
    0x00   // Index 7 - invalid
};

//---------------------------------------------------------------------
//  Commutation Update
//---------------------------------------------------------------------
void Get_Drive_Byte(void)
{
    // Read 3-bit hall sensor state from PORTE
    INT8U CurrentSensor = BLDC_SensorPORT & 0B00000111;

    // Select forward/reverse table based on direction pin
    if (BLDC_Direction == 0)
    {
        DriveByte = FwdTable[CurrentSensor];
    }
    else
    {
        DriveByte = RevTable[CurrentSensor];
    }
}

//---------------------------------------------------------------------
//  System Initialization
//---------------------------------------------------------------------
void Initialize(void)
{
    Get_Drive_Byte();            // Load initial drive state from
sensor position
    ADC_Result = 0x00;           // Initial speed = 0
    BLDC_DrivePort = 0x00;       // Disable all drive outputs
    BLDC_DrivePortTris = 0x00;   // Drive port set as output
    BLDC_SensorTRIS = 0xFF;      // Sensor port set as input

    TRISA = 0B00000011;          // RA0 = analog input, RA1 =
direction input

    // Timer0 configuration
    OPTION_REGbits.T0CS = 0;     // Timer0 clock = internal
instruction clock
    OPTION_REGbits.PSA = 0;      // Prescaler assigned to Timer0
    OPTION_REGbits.PS2 = 1;      // Prescaler = 1:256
    OPTION_REGbits.PS1 = 1;
    OPTION_REGbits.PS0 = 1;

    // ADC configuration
    ADCON1 = 0B00001110;         // Left-justified result, AN0 =
analog only
    ADCON0 = 0B11000001;         // ADC clock = internal RC, ADC
module enabled
    ADCON0bits.GO_nDONE = 1;     // Start first ADC conversion
```

```
}

//---------------------------------------------------------------------
//  ADC Speed Read
//---------------------------------------------------------------------
void ReadADC(void)
{
    // Wait for conversion complete, then read and restart
    if (ADCON0bits.GO_nDONE) return;

    ADC_Result = ADRESH;          // Read high 8 bits of result
    ADCON0bits.GO_nDONE = 1;      // Restart next conversion
}

//---------------------------------------------------------------------
//  Main Program
//---------------------------------------------------------------------
void main(void)
{
    Initialize();

    while (1)
    {
        ReadADC();  // Read speed potentiometer value

        // Software PWM: full speed at 0xFF, else modulate high-side
        if (ADC_Result != 0xFF)
        {
            // Turn off high-side drivers during PWM off-time
            if (TMR0 + ADC_Result <= 0xFF)
            {
                DriveByte &= BLDC_StopMask;
            }
        }

        // Output drive signal to motor
        BLDC_DrivePort = DriveByte;

        // Update commutation based on current hall sensor position
        Get_Drive_Byte();
    }
}
```